

EXPLORING COLLABORATION BETWEEN A LANGUAGE AND A NON-LANGUAGE AGENT

Harini S I*  Somesh Singh*    Yaman K Singla 

Rajiv Ratn Shah  David Doermann  Balaji Krishnamurthy 

 Adobe Media and Data Science Research (MDSR)

 IIT-Delhi,  SUNY at Buffalo

✉ behavior-in-the-wild@googlegroups.com

ABSTRACT

LLMs are increasingly deployed as orchestrators that coordinate specialized sub-agents to solve complex tasks through natural language. However, in many important domains like game playing and robotics the strongest available agents are not language models. Integrating non language agents with LLMs would require *verbalization* or compressing their rich continuous representations into sparse textual summaries at each interaction step. To study whether verbalization constitutes a bottleneck, we introduce LLAMIA-BENCH, a suite of six collaborative chess tasks spanning three facets: behavioral imitation, state assessment, and natural-language explanation. Tasks are drawn from prominent problems in chess literature and industry, each unsolvable by either component alone, and selected to vary in how much relevant signal survives verbalization. Our experiments reveal a consistent *verbalization debt*: the performance gap between verbalized and internalized integration widens throughout training and persists as the LLM scales from 4B to 14B parameters. To close this gap, we introduce *latent state internalization*, which projects the subagent’s continuous representations directly into the LLM’s token stream as learned state tokens, with dynamic re-encoding as actions advance the environment state. A single 14B model, LLAMIA, trained under this paradigm matches or exceeds dedicated task specialists and frontier models including GPT-5.1 with tool access across all benchmark tasks, and generalizes out-of-distribution where task-specific finetunes collapse.

1 INTRODUCTION

Large language models (LLMs) are increasingly deployed as general-purpose orchestrators that coordinate tools and agents to solve complex tasks (Hong et al., 2024; Tran et al., 2025). A central pattern in this paradigm is collaboration with *subagents*: specialized agents trained to excel within a narrow domain (Anthropic, 2025; OpenAI, 2025). This collaboration allows orchestrator LLMs to utilize the subagent’s domain specific intelligence (Hong et al., 2024) and preserve its context length by task delegation (Zhang et al., 2024b), enabling effective multi-step reasoning and planning over long horizons. Today, this collaboration is mediated entirely through natural language: the LLM invokes the subagent, receives a natural language description of its output, and reasons over that description to decide subsequent actions (Tran et al., 2025). Verbal collaboration is natural when both agents are language models as they share a vocabulary and can express their state in words. However, in many important domains such as game playing, robotics, and autonomous driving the strongest available agents are not language models; their expertise is encoded in internal representations: latent states capturing policy, value estimates, and learned features like AlphaZero (Silver et al., 2017) and RT-1 (Brohan et al., 2022). This mismatch between LLMs and non-language agents’ input space raises a fundamental question:

*Equal Contribution.

How can LLMs effectively collaborate and jointly reason with non-language subagents?

The depth of collaboration between LLMs and subagents can solve many useful tasks that neither can solve alone. Consider chess: LLMs have been trained on more chess literature than most experts will study in a lifetime, yet they cannot leverage it to play the game competently, trailing far behind modern engines and experts. Conversely, pretrained engines surpassed human grandmasters decades ago (Campbell et al., 2002), yet they remain narrow specialists that are unable to explain the rationale behind a move or strategize under different contexts. And there exists tasks like game commentary, preparing against an opponent, and designing interesting puzzles which require both, the chess engine’s deep positional understanding and the LLM’s ability to reason over human intent. Effective collaboration between LLMs and the subagent can unlock these applications.

Existing approaches to LLM-subagent collaboration attempt to bridge this gap symbolically, by *verbalizing* the subagent’s outputs into natural language before passing them to the LLM. Early work finetunes language models on textual descriptions of agent actions and value estimates (Zang et al., 2019; Lee et al., 2022), while more recent systems rely on in-context learning and tool-calling interfaces to surface subagent outputs at inference time (Schick et al., 2023; Yao et al., 2022; Kim et al., 2024). However, these approaches share a common assumption: that the subagent’s expertise can be faithfully verbalized. In this work we demonstrate that this assumption is fundamentally limiting. A chess engine’s latent representation encodes positional structures, long-range tactical motifs, and learned look-ahead (Jenner et al., 2024)—semantic features that cannot be translated to text faithfully. Verbalization therefore forces the subagent’s representations through a lossy bottleneck, collapsing rich latent structure into surface-level descriptions, we call this concept the **Verbalization Debt**. Moreover, this error compounds across interactions, in multi-step settings, each exchange between the agents strips away details, and accumulates errors over the reasoning horizon, diminishing the benefits that motivated this collaboration in the first place.

Internalization: To bridge this gap we introduce *latent state internalization*, a paradigm in which an LLM reasons over a non-language agent’s internal state over a single trace of **language**, **action**, and **latent state** tokens, where latent states are the agent’s penultimate-layer activations projected into the LLM’s embedding space. As shown in Figure 2, the paradigm consist of three distinct steps: ① the LLM reasons in language and generates actions that evolve the environment state, including counterfactual states within its CoT; ② The LLM on demand requests the agent’s for the evaluation of its current state; ③ the agent encodes the resulting state and its activations are projected into $k=32$ latent tokens appended to the reasoning trace, dynamically re-encoded after each state transition. We train a lightweight three-layer MLP, **LatentBridge** (Section 2.2), that learns the projection from the agent’s internal state to the LLM’s token space.

LLAMIA: We instantiate latent state internalization by training LLMs (Yang et al., 2025) in two stages: supervised projector alignment followed by reinforcement learning (DAPO), detailed in Section 2.3. We call the resulting model LLAMIA (Large Language and Action Models with Internal Agents). Beyond the technical contribution, internalization opens new avenues for real world applications. Because internalization requires access to model weights, it cannot be applied directly to closed-source models. LLAMIA resolves this tension by acting as a bridge: it interacts with LLMs in natural language and internalizes the subagent, giving closed-weight models indirect but faithful access to the non language’s expertise. This makes real-world creative applications like grounded game commentary and opponent-specific preparation deployable today, without retraining closed-weight models.

Verbalization Debt: To conclusively establish and analyze the effect of internalization, we train a controlled ablation, LLAMIA-Verb, in which the subagent’s outputs are exposed to the LLM through verbalization, without the latent tokens. LLAMIA consistently achieves higher reward across all tasks throughout training (Figure 3). The gap widens on tasks requiring deeper multi-step integration. These results show that verbalization is a fundamental bottleneck when non-language agents are treated as tools.

LLAMIA-Bench, Chess as a testbed: Despite abundant applications, LLM collaboration with non-language agents remains underexplored. A central reason is the lack of environments that combine what is needed to structurally test this collaboration: diverse tasks with verifiable metrics, large-scale public data, and open pretrained agents. Chess satisfies all three: decades of human-engine collaboration on commentary, preparation, and puzzles have produced established evaluation

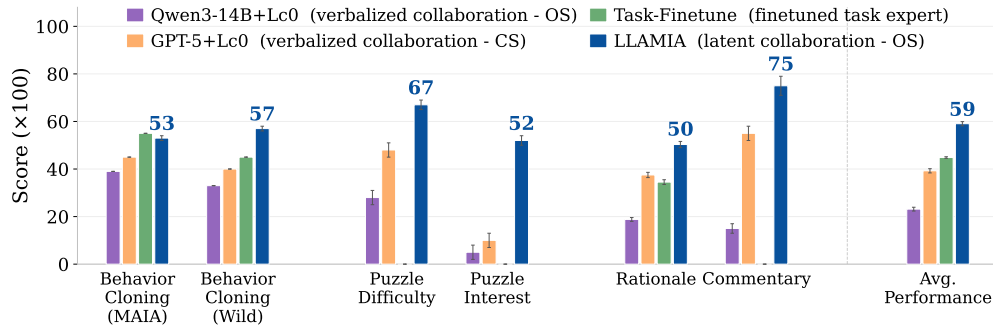


Figure 1: **LLAMIA-Bench performance across collaboration interfaces.** Scores ($\times 100$) on six chess-LLM tasks. Methods are labeled by access type and backbone (OS = open-source, CS = closed-source): verbalized tool use (Qwen3-14B+Lc0, GPT-5+Lc0), per-task finetuned experts (Task-Finetune), and LLAMIA’s latent integration. LLAMIA matches or exceeds every baseline on every task and is the only system to score on Puzzle Interest, where the engine signal has no text surrogate. “Avg. Performance” averages each method over its reported tasks.

protocols, large-scale public databases like Lichess, and annotated corpora [Feng et al. \(2024\)](#). We therefore use chess as our primary testbed and introduce LLAMIA-BENCH (Section B), a curated suite of six tasks spanning behavior cloning across skill levels, puzzle interest and difficulty estimation, move annotation and game-level commentary. We introduce a new dataset curated from [Agadator’s YouTube Channel](#) for game level commentary.

Results. A single LLAMIA-14B model matches or exceeds every task-specific specialist and frontier model across all six LLAMIA-BENCH tasks (Figure 1). The verbalization debt is sharpest on tasks requiring multi-step state tracking or non-verbalizable signals: LLAMIA-Verb’s reward stays flat on commentary and puzzle interest despite identical compute. Internalization also shapes the *kind* of collaboration that emerges: LLAMIA develops counterfactual queries and multi-step lookahead strategies absent from LLAMIA-Verb, which collapses to engine-follow regardless of task, suggesting that access to the full latent state is what makes richer collaboration learnable. Beyond benchmark numbers, LLAMIA reproduces human behavioral signatures: under time pressure it commits the same blunders humans do, and at different skill levels its attention concentrates on the same pieces human players prioritize. A human study confirms this: LLAMIA’s gameplay passes as human in the majority of trials, and its commentary is preferred on both strategic insight and explanatory depth compared to the verbalized baseline.

Our contributions are fourfold:

1. **Latent state internalization** (Section 2): A paradigm for LLM non language agent collaboration, enabling LLMs to reason over interleaved chain of `<language, action, latent state>` tokens.
2. **LLAMIA** (Section 2): A two-stage training framework of self-supervised projector alignment followed by end-to-end RL (DAPO) that yields a single model, LLAMIA, achieving state-of-the-art performance across diverse collaborative tasks. LLAMIA enables real world application by giving close-weight models faithful access to subagents.
3. **Verbalization Debt** (Section 3): Through controlled ablations against LLAMIA-Verb, we define and quantify the *Verbalization Debt* (the performance gap between internalized and verbalized integration), showing it is suboptimal in both performance and compute, with the gap widening on tasks requiring deeper or multi-step collaboration.
4. **LLAMIA-BENCH** (Section 3): A curated benchmark of six chess tasks spanning diverse collaborative tasks including behavior cloning, puzzle understanding, commentary, and planning.

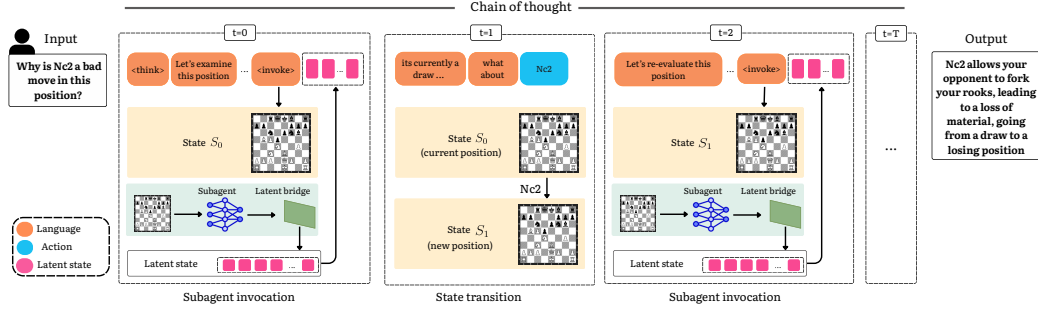


Figure 2: **Latent State Internalization.** A single chain of thought rolled out over time. $t=0$ (*subagent invocation*): the LLM begins reasoning in **language** and emits `<invoke>`; the current board S_0 is passed through the frozen subagent (Lc0-BT4) and the **LatentBridge** maps its hidden activations into $k=32$ continuous **latent state** tokens that are appended to the context. $t=1$ (*state transition*): conditioned on S_0 and its latent state, the LLM continues reasoning and emits an **action** (Nc2), which advances the environment to S_1 . $t=2$ (*subagent invocation*): the LLM chooses to re-invoke, encoding S_1 into a fresh **latent state** so subsequent reasoning is grounded in the updated board. The LLM decides when to re-invoke rather than re-encoding on every step. Stage 1 trains only the LatentBridge on state-policy pairs; Stage 2 jointly fine-tunes it with the LLM via DAPO. The subagent is frozen throughout.

2 METHODOLOGY

2.1 FORMULATION

We formalize latent state internalization through a running example. An LLM plays chess with access to a pretrained engine exposed through a **tool API**: functions to read the board state and legal moves, advance the game by making moves, and—critically—query the engine’s assessment of any position via `get_policy` (full schema in Section H.4.3). The first two categories handle environment interaction; `get_policy` is the interface to the subagent, and *what it returns* is the variable this paper studies.

“The knight should develop. Nf6. `get_policy(s1)` → ( ) Nc2: P=0.34, +0.12; e4: P=0.21, +0.08; ... ”

The LLM reasons in language, plays a knight move to f6 (advancing the game to position s_1), then calls `get_policy`. The engine returns a text summary—per-move prior probabilities and value estimates—that both model variants receive. In LLAMIA, the call additionally injects k continuous **state tokens** () projected from the engine’s internal activations. In LLAMIA-Verb, only the text is returned. Three token types thus interleave in a trace: **language tokens** (the LLM’s chain-of-thought), **action tokens** (moves that advance the game), and **state tokens** (the engine’s projected latent representation, present only under internalization).

More formally: a policy π_θ (the LLM) interacts with an environment whose states $s \in \mathcal{S}$ evolve through actions $a \in \mathcal{A}$. A pretrained **subagent** with encoder $G_\psi: \mathcal{S} \rightarrow \mathbb{R}^d$ maps each state to a latent representation $h_s = G_\psi(s)$. The LLM queries the subagent at self-chosen moments via `get_policy`, specifying either the current position s_t or a hypothetical state reached by a candidate move. Over a T -step interaction the policy produces a trace τ :

$$\tau = (z_0^{1:k}, w_{1:j_1}, a_1, z_1^{1:k}, \dots, w_{\text{final}}) \quad (1)$$

where $w_t \in \mathcal{V}$ are language tokens (including the text returned by `get_policy`), a_t are actions, and $z_t^{1:k}$ are k contiguous state tokens injected alongside the text response. In LLAMIA-Verb, $k=0$: no state tokens appear, and the LLM reasons over text alone. In LLAMIA, $k=32$. How these state tokens are produced defines the internalization method.

Verbalization. Every `get_policy` call returns a text serialization of the subagent’s output:

$$\hat{w}_t = \text{Verb}(G_\psi(s_t)) \in \mathcal{V}^* \quad (2)$$

Both LLAMIA and LLAMIA-Verb receive $\hat{w}_t$. A typical return lists the engine’s top moves with prior probabilities and value estimates. This captures the subagent’s headline assessment but discards

the remaining structure in $G_\psi(s_t)$: the full distribution over all legal moves, the value landscape across candidate continuations, and positional features like piece coordination and king safety that interpretability work has identified in engine hidden layers Jenner et al. (2024).

Internalization. In LLAMIA, each `get_policy` call additionally produces k continuous tokens by projecting the subagent’s full latent state into the LLM’s embedding space via a learned projection, **LatentBridge**:

$$z_t = H_\varphi(G_\psi(s_t)) \in \mathbb{R}^{k \times e} \quad (3)$$

$H_\varphi: \mathbb{R}^d \rightarrow \mathbb{R}^{k \times e}$ produces k continuous embeddings of dimension e matching the LLM’s hidden size. These state tokens sit alongside language and action tokens in the trace (Figure 2), and the LLM attends over all three types jointly. Where the text serialization imposes a fixed summary regardless of what the current reasoning step requires, latent tokens let the LLM’s attention selectively read different aspects of the representation at each step. Gradients flow from the training objective through the LLM back to H_φ , so the projection adapts to the task.

2.2 ARCHITECTURE

Subagent. We instantiate G_ψ with Lc0-BT4 Monroe & Chalmers (2024), the strongest open-source chess engine, a 15-layer Transformer encoder (240 M parameters) whose representations encode positional features, piece-value geometry, and lookahead-related structure Jenner et al. (2024), producing $h_s \in \mathbb{R}^{1024}$. We ablate this choice across five Lc0 variants of varying strength in Appendix E.2.

Large Language Model. We use Qwen3 Yang et al. (2025), the strongest open-weight LLM at this scale at the time of training, as the backbone π_θ . The tool API (Section H.4.3) is provided in the system prompt via Hermes-format function calling; Qwen3 natively supports structured tool calls without additional training. To accept **state tokens**, k contiguous positions in the input sequence serve as placeholders whose embeddings are replaced by the projected state z_s .

LatentBridge. The projection $H_\varphi: \mathbb{R}^d \rightarrow \mathbb{R}^{k \times e}$ (Eq. 3) is a three-layer MLP with GeLU activations, following the projector design in vision-language models Liu et al. (2023a). It maps Lc0’s 1024-dimensional latent state into $k=32$ embeddings of dimension e matching the LLM’s hidden size. The resulting input to the LLM is a mixed sequence $[w_1, z_s^{1:k}, a_1, w_2]$ (Figure 2).

2.3 TRAINING

Training proceeds in two stages. Stage 1 aligns the subagent’s representations with the LLM’s embedding space while keeping the LLM frozen. Stage 2 trains both the LLM and LatentBridge jointly via reinforcement learning. The subagent remains frozen throughout.

2.3.1 STAGE 1: PROJECTOR ALIGNMENT

We train only H_φ while keeping π_θ frozen, on a dataset $\mathcal{D}_{\text{pre}} = \{(s, \pi(s))\}$ of state-policy pairs from the subagent’s self-play. Each example pairs the projected state tokens z_s with a language prompt (e.g., “Analyze position: top move?”; format in Section H.1), and the model learns to generate the correct action token via cross-entropy:

$$\mathcal{L}_{\text{Stage 1}} = -\mathbb{E}_{(s, \pi(s)) \sim \mathcal{D}_{\text{pre}}} \log \pi_\theta(\pi(s) \mid z_s, \text{prompt})$$

Because the LLM is frozen, the projector trains on abundant agent-generated data without risking catastrophic forgetting of language capabilities.

2.3.2 STAGE 2: REINFORCEMENT LEARNING

Stage 2 unfreezes both π_θ and H_φ and optimizes them jointly via DAPO Yu et al. (2025), a group-relative policy optimization variant with asymmetric clipping. Rollouts produce complete traces τ (Eq. 1). The policy gradient is computed over positions where the LLM generates: **language tokens** (index set $\mathcal{I}_L$) and **action tokens** (index set $\mathcal{I}_A$), collectively $\mathcal{I}_{\text{gen}} = \mathcal{I}_L \cup \mathcal{I}_A$. **State token** positions are agent-injected and excluded via gradient masking. Writing y_t for the token at position t , the importance-sampling ratio between the current policy π_θ and the reference policy $\pi_{\theta_{\text{old}}}$ from the

previous iteration depends on the integration channel:

$$r_t^{\text{verb}} = \frac{\pi_\theta(y_t \mid y_{<t}, \hat{w}_{<t})}{\pi_{\theta_{\text{old}}}(y_t \mid y_{<t}, \hat{w}_{<t})}, \quad (4)$$

$$r_t^{\text{latent}} = \frac{\pi_\theta(y_t \mid y_{<t}, \hat{z}_{<t})}{\pi_{\theta_{\text{old}}}(y_t \mid y_{<t}, \hat{z}_{<t})}. \quad (5)$$

The DAPO objective maximizes:

$$J(\theta, \varphi) = \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[\frac{1}{|\mathcal{I}_{\text{gen}}|_t} \sum_{t \in \mathcal{I}_L \cup \mathcal{I}_A} \min(r_t \hat{A}_t, \text{clip}(r_t, 1-\varepsilon_l, 1+\varepsilon_h) \hat{A}_t) \right] \quad (6)$$

where r_t is the importance ratio (Eq. 5), $\hat{A}_t = (\mathcal{R}(\tau) - \mu_G)/\sigma_G$ is the group-normalized advantage computed over G rollouts sharing the same prompt [Yu et al. \(2025\)](#), and the asymmetric clip bounds $\varepsilon_l \leq \varepsilon_h$ encourage exploration. Each LLAMIA-BENCH task defines a scalar outcome reward $\mathcal{R}(\tau)$ based on its evaluation metric (Appendix B).

Because H_φ is jointly optimized, the RL objective shapes the integration interface end-to-end: the projector learns what representation to present, while the LLM learns how to reason over it. The tool call itself is part of the policy’s action space, so RL also learns *when* and *whether* to query the subagent.

3 EXPERIMENTS AND RESULTS

We evaluate LLAMIA on LLAMIA-BENCH, a suite of six tasks spanning three collaboration facets: behavioral imitation, state assessment, and natural-language explanation (Section B). For comparing LLAMIA with closed source models where training isn’t possible, we pair them with Lc0 as a verbalized tool. To evaluate against open-source models we create three settings: an untrained tool-use baseline, a training-matched verbalization system, and our internalized model, isolating the integration interface as the sole variable. Within the open-source trained systems, we report both SFT and SFT + DAPO checkpoints to disentangle the contributions of supervised pretraining and reinforcement learning.

3.1 LLAMIA-BENCH

LLAMIA-BENCH spans six tasks drawn from prominent problems in chess literature and industry, each unsolvable by either component alone: the subagent produces no language, while the LLM lacks the positional signals that make chess-specific judgments possible. The suite covers behavior cloning, puzzle understanding, move annotation, and game-level commentary (details in Section B). Behavior cloning, difficulty estimation, and move annotation are drawn from established benchmarks [McIlroy-Young et al. \(2020\)](#); [Lichess.org \(2024b\)](#); [Jhamtani et al. \(2018\)](#). We introduce three new evaluation targets: **Wild BC**, three OOD splits (GM-25, Low-Time, ΔElo) probing generalization to grandmaster play, time pressure, and asymmetric skill-gap; **interest estimation**, ranking puzzles by community-derived interestingness scores, a signal with no verbal proxy in any engine output; and **Agadmator-2K**, the first large-scale game-level commentary dataset of 1,900 narrated games with move-aligned transcripts. Full dataset descriptions, metrics, and per-task prompts appear in Appendices B–H.

3.2 SETUP

Baselines We compare four systems. (1) **GPT-5.1 + Lc0**: the strongest frontier model with verbalized Lc0-BT4 tool access. (2) **Qwen3-14B + Lc0**: the LLAMIA backbone with the same verbalized tool and 5-shot prompting, without training. (3) **LLAMIA-Verb**: the matched-recipe ablation with the latent channel replaced by the verbalized tool, isolating the integration interface as the sole variable. (4) **LLAMIA**: our full system with latent state-token integration. Extended comparisons including LLM-only baselines, SFT and DAPO checkpoints at all three model sizes (4B, 8B and 14B), frontier model comparisons, and task-specific experts are in Section D. Training details and hyperparameters are in Appendix A.

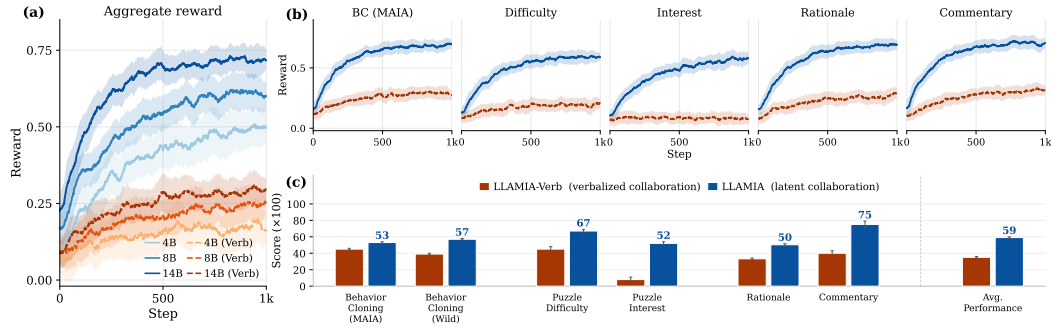


Figure 3: DAPO training dynamics and LLAMIA-Bench evaluation. (a) Aggregate reward vs. training step at three backbone scales. Solid: LLAMIA (latent); dashed: LLAMIA-Verb (verbalized, identical recipe and backbone). The debt widens throughout and reaches 2–3 $\times$ by convergence; scaling the backbone does not close it for LLAMIA-Verb. (b) Per-task reward curves at 14B. LLAMIA-Verb gains partial signal on behavior cloning and difficulty (tasks with verbalizable proxies) but stays near-flat on interest and commentary, where the reward requires non-verbalizable features or multi-step integration. (c) LLAMIA-Bench scores (0–100). Puzzle Interest is diagnostic: every verbalized system scores ≤ 12 regardless of model scale or frontier capability, while LLAMIA-14B reaches 52. Full per-task tables with confidence intervals in Section D.

Metrics Each task has a primary metric detailed in Section B: move-matching accuracy (behavior cloning), Spearman ρ (difficulty and interest), and G-eval and BLEU-2 (annotation and commentary). Primary metrics include 95% bootstrap confidence intervals where sample sizes warrant; per-task breakdowns with CIs appear in Section D.

3.3 MAIN RESULTS

LLAMIA-14B achieves the highest score on all six LLAMIA-BENCH tasks (Figure 3c), surpassing frontier verbalized systems an order of magnitude larger and remaining competitive with dedicated task-specific finetunes that are trained on substantially more in-domain data. On behavior cloning, Maia and Allie are trained on tens of millions of chess-specific games, against LLAMIA’s general-purpose backbone. LLAMIA-14B remains inside the expert band on the in-distribution Maia split and surpasses the strongest expert by a wide margin on the OOD Wild splits (GM-25, Low-Time, Δ Elo), which probe regimes absent from the experts’ blitz-only training mixture. Latent access to the engine’s policy and value structure thus generalizes more reliably than direct supervision on a narrower distribution. The advantage over GPT-5 + Lc0 does not require the 14B backbone: LLAMIA-8B already leads on all six tasks, and LLAMIA-4B on four of six (Table 15). Per-task evaluations with additional metrics, baselines, and confidence intervals are in Section D.

Latent tokens enable new evaluation targets. Puzzle Interest requires ranking positions by community-derived interestingness, a signal that depends on the engine’s policy distribution and value gradients across candidate moves. No verbalized engine output carries these features. Every verbalized system scores ≤ 12 on Interest regardless of model scale or frontier capability; LLAMIA-14B reaches 52 (Figure 3c). Verbalization has zero useful signal for this task, while latent tokens give the LLM direct access to the distributional structure that defines interestingness.

3.4 VERBALIZATION DEBT

LLAMIA and LLAMIA-Verb share the same 14B backbone, Lc0-BT4 subagent, and DAPO recipe; the only difference is whether the subagent’s state reaches the LLM as latent tokens or as verbalized text. We define the resulting performance gap as the **verbalization debt**.

Verbalization debt is significant across all tasks Figure 3 shows that the verbalization debt is consistent across all tasks. The debt varies across tasks, it is largest on tasks whose target signal lives in the engine’s full policy distribution or value landscape (Interest, Commentary), which have no

	LLAMIA (Internalization)					LLAMIA-VERB (Verbalization)				
	Follow	Override	CF-Query	Multi-step	Abstain	Follow	Override	CF-Query	Multi-step	Abstain
Gameplay	65	5	14	2	14	76	4	1	—	19
Behav. Cloning	14	48	20	12	6	65	19	2	—	14
Commentary	10	12	40	32	6	62	17	4	2	15
Puzzle (Diff.)	45	10	24	10	11	72	9	2	—	17
Puzzle (Int.)	52	8	18	5	17	70	7	2	1	20
Average	37	17	23	12	11	69	11	2	1	17

Figure 4: **Collaboration strategy distribution (%) per task at convergence.** Fraction of 500 episodes assigned to each strategy by a GPT-4o judge ($\kappa=0.78$). *Left:* LLAMIA (latent). *Right:* LLAMIA-Verb (verbalized). LLAMIA’s dominant strategy shifts with the task (engine-follow for gameplay, consult-then-override for BC, counterfactual query for commentary); LLAMIA-Verb collapses to engine-follow on every row (62–76%). Detailed discussion can be found in Section E.1

faithful text equivalent. It is smallest on in-distribution behavior cloning, where the engine’s top- k moves already approximate the answer and the verbal summary loses little.

The debt persists across backbone scale. Increasing the LLM from 4B to 14B improves both systems, but the debt persists at every scale (Table 15). On Interest, LLAMIA-Verb-14B scores 8 while LLAMIA-4B already reaches 38.

The debt widens throughout training. The debt grows throughout DAPO, reaching 2–3 $\times$ by the end of training (Figure 3a). Per-task reward curves (Figure 3b) reveal where the verbal channel saturates: LLAMIA-Verb gains partial signal on behavior cloning and difficulty, where the verbalized output carries a useful proxy (top- k moves, solution length), but stays near-flat on interest and commentary, where no such proxy exists.

3.5 COLLABORATION STRATEGIES

Does the integration interface determine how the model learns to use the subagent, or only how well it performs? We classify subagent invocations during evaluation into five recurring strategies and trace their evolution during DAPO training (Figure 7; strategy definitions and per-task heatmaps in Figure 6).

The five strategies are *engine-follow* (adopt the top recommendation), *consult-then-override* (query then diverge), *counterfactual query* (play a hypothetical move, re-invoke, compare states), *multi-step lookahead* (chain two or three counterfactual sequences), and *abstention* (act from language knowledge alone). A GPT-4o judge classifies 500 episodes per task per system ($\kappa=0.78$ vs. human raters).

Internalization produces task-specific collaboration; verbalization collapses it. LLAMIA adapts its strategy to the task: engine-follow dominates gameplay (65%), consult-then-override dominates behavior cloning (48%), and counterfactual query dominates commentary (40%). LLAMIA-Verb collapses to engine-follow on every task (62–76%), regardless of what the task requires (Figure 6). The verbalized channel returns the same compressed summary no matter how the model queries it, so RL converges on a single use pattern.

3.6 HUMAN EVALUATION

Gameplay. Skilled players ($n=12$, all ≥ 1700 Elo) cannot reliably distinguish LLAMIA from a human opponent: detection falls below chance (Section 3.6), and post-game ratings place LLAMIA

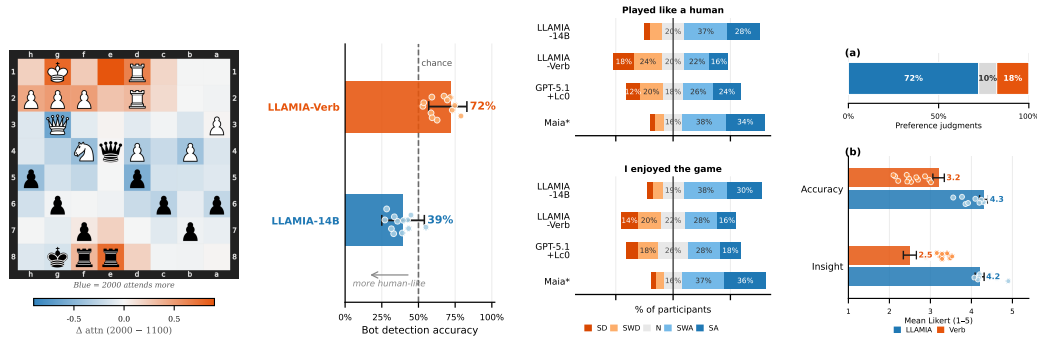


Figure 5: **Human evaluation** ($n=12$ skilled players, all rated ≥ 1700). (a) Attention difference over latent tokens on a back-rank mate position: conditioning on 2000 Elo concentrates attention on mating geometry (red); 1100 Elo disperses to material (blue). The same latent state is read differently depending on the language instruction. (b) Bot-detection rate (Study 1): LLaMIA-14B passes as human in 61% of trials (detection 39%, below chance); LLaMIA-Verb detected in 72%. (c) Post-game Likert (Strongly Disagree → Strongly Agree): LLaMIA matches Maia* (best Maia variant per Elo bucket) on human-likeness without training on human-move distributions. (d) Commentary preference (Study 2): 72.2% of 180 judgments favour LLaMIA; Insight gap (1.7 pts) exceeds Accuracy gap (1.1 pts). Study design and qualitative results in Section F.

alongside Maia* on perceived human-likeness (Section 3.6). Maia is trained directly on millions of move distributions; LLaMIA receives no human-move supervision. Instead, behavioral signatures such as time-pressure blunders and skill-appropriate piece saliency emerge from latent-state conditioning alone. LLaMIA-Verb, trained with the same backbone and DAPO budget, is detected at rates well above chance, consistent with the stylistic regularities that verbal summaries impose on move selection.

Commentary. Both systems achieve comparable factual accuracy: material balance, initiative assessment, and basic evaluations survive verbalization reasonably well. The gap concentrates on strategic insight (Section 3.6), where participants rate LLaMIA higher by a wider margin on the Insight dimension than on Accuracy. Text preserves *what* is happening on the board, but explaining *why* a move is strong requires representational features (policy gradients, value topology, look-ahead depth) that do not survive verbal compression.

Latent tokens are instruction-modulated. On a fixed back-rank mate position (Section 3.6), LLaMIA’s attention over the latent tokens shifts with the target Elo: at 2000 it concentrates on the mating geometry, at 1100 it disperses to material. The latent representation is identical in both cases; what changes is the LLM’s reading, conditioned on the natural-language Elo instruction. The projected state functions as a perceptual input shaped by task context, not a static feature vector.

4 RELATED WORK

The dominant paradigm for LLM-agent integration is text-mediated: ReAct Yao et al. (2022), Toolformer Schick et al. (2023), and multi-agent orchestrators like AutoGen Wu et al. (2023) and HuggingGPT Shen et al. (2024) all route communication through natural language. This is lossless when both parties are language models, but compresses the policy and value representations of pretrained neural agents into a few tokens. Latent communication channels exist in multi-agent RL Sukhbaatar et al. (2016); Das et al. (2019); Foerster et al. (2016), but assume homogeneous populations trained jointly from scratch, not a frozen LLM paired with a frozen specialist. Cross-modal injection (PaLM-E Driess et al. (2023), RT-2 Brohan et al. (2023)) and latent recurrence within a single model (CoCoNut Hao et al. (2025)) are architecturally closer but project raw sensory observations rather than the processed internal representations of a pretrained agent. On the domain side, neural chess engines encode rich positional structure in their activations Silver et al. (2017); Monroe & Chalmers (2024), as interpretability work confirms Jenner et al. (2024), yet prior LLM-

chess work either trains task-specific models [Jhamtani et al. \(2018\)](#) or conditions on verbalized outputs [Feng et al. \(2024\)](#); none exposes the engine’s latent state to the LLM.

5 CONCLUSION

We introduced latent state internalization, an LLM–agent integration method that replaces verbalized communication with direct projection of the agent’s continuous representations into the LLM’s embedding space. A single LLAMIA-14B, trained via projector alignment followed by DAPO, matches or exceeds dedicated task finetunes across LLAMIA-BENCH. The verbalization debt widens with interaction depth and on signals that resist text serialization, and does not close with LLM scale or RL budget in our evaluated range, establishing verbalization as a structural bottleneck for LLM–agent collaboration.

REFERENCES

- Anthropic. Create custom subagents. <https://code.claude.com/docs/en/sub-agents>, 2025. Claude Code Documentation. Accessed: 2026-05-02. [1](#)
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022. [1](#)
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, Pete Florence, Chuyuan Fu, Montse Gonzalez Arenas, Keerthana Gopalakrishnan, Kehang Han, Karol Hausman, Alexander Herzog, Jasmine Hsu, Brian Ichter, Alex Irpan, Nikhil Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Lisa Lee, Tsang-Wei Edward Lee, Sergey Levine, Yao Lu, Henryk Michalewski, Igor Mordatch, Karl Pertsch, Kanishka Rao, Krista Reymann, Michael Ryoo, Grecia Salazar, Pannag Sanketi, Pierre Sermanet, Jaspiar Singh, Anikait Singh, Radu Soricut, Huong Tran, Vincent Vanhoucke, Quan Vuong, Ayzaan Wahid, Stefan Welker, Paul Wohlhart, Jialin Wu, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. Rt-2: Vision-language-action models transfer web knowledge to robotic control, 2023. URL <https://arxiv.org/abs/2307.15818>. [9](#)
- Caissabase Contributors. Caissabase: A free chess database. <https://mattplayschess.com/free-large-db/>, 2024. Accessed: 2025. [18](#)
- Murray Campbell, A Joseph Hoane Jr, and Feng-hsiung Hsu. Deep blue. *Artificial intelligence*, 134 (1-2):57–83, 2002. [2](#)
- Abhishek Das, Théophile Gervet, Joshua Romoff, Dhruv Batra, Devi Parikh, Mike Rabbat, and Joelle Pineau. Tarmac: Targeted multi-agent communication. In *International Conference on machine learning*, pp. 1538–1546. PMLR, 2019. [9](#)
- Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, Andy Zeng, Igor Mordatch, and Pete Florence. Palm-e: An embodied multimodal language model, 2023. URL <https://arxiv.org/abs/2303.03378>. [9](#)
- Xidong Feng, Yicheng Luo, Ziyang Wang, Hongrui Tang, Mengyue Yang, Kun Shao, David Mguni, Yali Du, and Jun Wang. Chessgpt: Bridging policy learning and language modeling. *Advances in Neural Information Processing Systems*, 36, 2024. [3](#), [10](#)
- Jakob Foerster, Ioannis Alexandros Assael, Nando De Freitas, and Shimon Whiteson. Learning to communicate with deep multi-agent reinforcement learning. *Advances in neural information processing systems*, 29, 2016. [9](#)
- GameKnot. GameKnot: Online chess. <https://gameknot.com/>, 2024. Accessed: 2024. [38](#)
- Mark E Glickman. Example of the glicko-2 system. *Boston University*, 28:2012, 2012. [20](#)

- Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space, 2025. URL <https://arxiv.org/abs/2412.06769>. 9
- Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework, November 2024. URL <http://arxiv.org/abs/2308.00352>. arXiv:2308.00352 [cs]. 1
- Erik Jenner, Shreyas Kapur, Vasil Georgiev, Cameron Allen, Scott Emmons, and Stuart Russell. Evidence of learned look-ahead in a chess-playing neural network. *arXiv preprint arXiv:2406.00877*, 2024. 2, 5, 9, 18
- Harsh Jhamtani, Varun Gangal, Eduard Hovy, Graham Neubig, and Taylor Berg-Kirkpatrick. Learning to generate move-by-move commentary for chess games from large-scale social forum data. In Iryna Gurevych and Yusuke Miyao (eds.), *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1661–1671, Melbourne, Australia, July 2018. Association for Computational Linguistics. doi: 10.18653/v1/P18-1154. URL <https://aclanthology.org/P18-1154>. 6, 10, 18, 19, 21, 25
- Jaechang Kim, Jinmin Goh, Inseok Hwang, Jaewoong Cho, and Jungseul Ok. Bridging the gap between expert and language models: Concept-guided chess commentary generation and evaluation. *arXiv preprint arXiv:2410.20811*, 2024. 2, 26
- Andrew Lee, David Wu, Emily Dinan, and Mike Lewis. Improving chess commentaries by combining language models with symbolic reasoning engines. *arXiv preprint arXiv:2212.08195*, 2022. 2, 21
- Lichess.org. Lichess open database: Puzzles. <https://database.lichess.org/>, 2024a. Accessed: 2025. 18
- Lichess.org. Lichess open database: Puzzles. <https://database.lichess.org/#puzzles>, 2024b. Accessed: 2025. 6, 18, 19
- Bo Liu, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone. Llm+ p: Empowering large language models with optimal planning proficiency. *arXiv preprint arXiv:2304.11477*, 2023a. 5
- Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-eval: Nlg evaluation using gpt-4 with better human alignment. *arXiv preprint arXiv:2303.16634*, 2023b. 21
- Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-eval: Nlg evaluation using gpt-4 with better human alignment, 2023c. URL <https://arxiv.org/abs/2303.16634>. 34
- Reid McIlroy-Young, Siddhartha Sen, Jon Kleinberg, and Ashton Anderson. Aligning superhuman ai with human behavior: Chess as a model system. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 1677–1687, 2020. 6, 18, 19, 20, 21, 23, 24
- Szymon Miłosz and Paweł Kapusta. Predicting chess puzzle difficulty with transformers. In *2024 IEEE International Conference on Big Data (BigData)*, pp. 8377–8384. IEEE, 2024. 23
- Daniel Monroe and Philip A Chalmers. Mastering chess with a transformer model. *arXiv preprint arXiv:2409.12272*, 2024. 5, 9, 17
- OpenAI. Subagents – Codex. <https://developers.openai.com/codex/subagents>, 2025. OpenAI Developer Documentation. Accessed: 2026-05-02. 1
- Soroush Saghaian and Lihi Idan. Effective generative ai: The human-algorithm centaur. *Harvard Data Science Review*, (Special Issue 5), 2024. 34

- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023. URL <https://arxiv.org/abs/2302.04761>. 2, 9
- Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. *Advances in Neural Information Processing Systems*, 36, 2024. 9
- David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of go without human knowledge. *nature*, 550(7676):354–359, 2017. 1, 9
- Sainbayar Sukhbaatar, Arthur Szlam, and Rob Fergus. Learning multiagent communication with backpropagation, 2016. URL <https://arxiv.org/abs/1605.07736>. 9
- Khanh-Tung Tran, Dung Dao, Minh-Duong Nguyen, Quoc-Viet Pham, Barry O’Sullivan, and Hoang D. Nguyen. Multi-Agent Collaboration Mechanisms: A Survey of LLMs, January 2025. URL <http://arxiv.org/abs/2501.06322>. arXiv:2501.06322 [cs]. 1
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation framework. *arXiv preprint arXiv:2308.08155*, 2023. 9
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. 2, 5, 16, 21
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022. 2, 9, 21
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025. 5, 6, 17
- Hongyu Zang, Zhiwei Yu, and Xiaojun Wan. Automated chess commentator powered by neural chess engine. *arXiv preprint arXiv:1909.10413*, 2019. 2, 18, 23, 25, 28
- Yiming Zhang, Athul Paul Jacob, Vivian Lai, Daniel Fried, and Daphne Ippolito. Human-aligned chess with a bit of search, 2024a. URL <https://arxiv.org/abs/2410.03893>. 23, 24, 28
- Yusen Zhang, Ruoxi Sun, Yanfei Chen, Tomas Pfister, Rui Zhang, and Sercan Ö Arik. Chain of Agents: Large Language Models Collaborating on Long-Context Tasks, June 2024b. URL <http://arxiv.org/abs/2406.02818>. arXiv:2406.02818 [cs]. 1

A	Implementation Details	16
A.1	Libraries and Software	16
A.2	Model Architecture and LatentBridge	16
A.2.1	Backbone LLM	16
A.2.2	LatentBridge Projector	16
A.3	Training Configuration	16
A.3.1	Stage 1: Projector Alignment	17
A.3.2	Stage 2: Reinforcement Learning (DAPO)	17
A.4	Chess Specialist: Lc0-BT4	17
A.4.1	Engine Configuration	17
B	Datasets & Benchmarks	17
B.1	Dataset & Metrics	18
B.2	Detailed Task Descriptions	18
B.2.1	Game-Level Commentary	18
B.2.2	Behavior Cloning	19
B.2.3	Puzzle Understanding	20
B.2.4	Move Annotation	21
B.3	Data Contamination Statement	21
C	Baselines	21
C.1	Frontier LLMs with Verbalized Tools	21
C.2	LLAMIA-Verb	22
C.3	LLAMIA (4B, 8B, 14B)	22
C.4	Dedicated task finetunes	22
D	Extended Results	23
D.1	Behavior Cloning	23
D.2	Move Annotation & Game Commentary	23
D.2.1	Move Annotation	23
D.2.2	Game commentary	23
D.3	Puzzle Understanding	25
D.3.1	Difficulty and Interest prediction	26
D.4	Full LLAMIA-Bench Results Table	26
E	Ablations	28
E.1	Emergent Collaboration Agency	28
E.2	Agent Size and Playing Strength	29
E.2.1	LLM Backbone and Agent Strength	30
E.3	Projection Token Count	31

F	Human Evaluation Studies	31
F.1	Participants	32
F.2	Sensitivity Calibration and Participant Selection	32
F.3	Study 1: Gameplay Identification	32
F.4	Study 2: Commentary Quality	33
F.5	Results	34
G	Dataset Construction	36
G.1	Stage 1: Projector Alignment Data	36
G.2	Stage 2: Task-Specific RL Data	37
H	Prompts and Templates	38
H.1	Stage 1: Projector Alignment	38
H.2	Stage 2: DAPO System Prompt	38
H.3	<invoke> Trigger Format	39
H.4	Inference Harness and Tool-Call Protocol	40
H.4.1	Agent State	40
H.4.2	System Prompt	40
H.4.3	Tool Definitions	40

APPENDIX TABLE OF CONTENTS

A IMPLEMENTATION DETAILS

A.1 LIBRARIES AND SOFTWARE

Package	Version	Role
PyTorch	2.8	Training backend
MegatronLM	0.15.0	Tensor-parallel training
VERL	0.7.1	RL training framework
vLLM	0.10.2	Rollout inference engine
Ray	2.55.1	Distributed orchestration
Transformers	4.56.2	Model loading & tokenization
lc0	0.32.1	Chess specialist engine (UCI)

Table 1: Key software dependencies.

A.2 MODEL ARCHITECTURE AND LATENTBRIDGE

A.2.1 BACKBONE LLM

Model	Params	Hidden	Layers	Heads (KV)	Context
Qwen3-4B	4B	2,560	36	32 (8)	32,768
Qwen3-8B	8B	4,096	36	32 (8)	32,768
Qwen3-14B	14B	5,120	40	40 (8)	32,768

Table 2: Backbone LLM architectures. All variants use GQA with 8 KV heads. Chain-of-thought reasoning is disabled (`enable_thinking=False`) following [Yang et al. \(2025\)](#). `max_model_len` is set to 32,768 tokens shared across system prompt, history, tool I/O, and response; `MAX_ROUNDS=20` caps LLM calls per query.

A.2.2 LATENTBRIDGE PROJECTOR

Parameter	Chess (Lc0-BT4)
Input dimension d	1024
Output tokens k	32
MLP layers	3
Activation	GeLU
Source layer	14 of 15 (penultimate)

Table 3: LatentBridge projector H_φ configuration. H_φ is a three-layer MLP with GeLU activations mapping the subagent’s residual stream into $k=32$ tokens of dimension e matching the LLM’s hidden size (see Section E.3 for the k ablation). A special `<state>` token anchors each injection site; k contiguous positions immediately following it are overwritten with $\mathbf{z}_t = H_\varphi(\mathbf{h}_{s_t})$ before the LLM forward pass.

A.3 TRAINING CONFIGURATION

Model parameters and optimizer state are partitioned across 4 nodes of 8xA100 GPUs using PyTorch FSDP, coordinated via Ray. The rollout vLLM instance and lc0 server fleet co-reside on the same GPUs, with 3 GiB per GPU reserved for the BT4 network.

A.3.1 STAGE 1: PROJECTOR ALIGNMENT

Stage 1 trains H_φ alone with F_θ frozen on 5 M state-policy pairs drawn from lc0’s forward pass over the Lichess evaluation database. Each example pairs a FEN with lc0’s top moves and pawn evaluations; the model minimizes cross-entropy over verbalized policy output conditioned on $z_s = H_\varphi(\mathbf{h}_s)$ and a fixed prompt template (Section H). Skipping Stage 1 causes training instability during the first 40% of Stage 2 (Section E).

Hyperparameter	Value
LR (H_φ)	2e-4
Batch size	256
Steps	2 epochs (≈ 39 K steps)
k (state tokens)	32
Max sequence length	32,768

Table 4: Stage 1 (projector alignment) hyperparameters. F_θ is frozen.

A.3.2 STAGE 2: REINFORCEMENT LEARNING (DAPO)

Stage 2 fine-tunes both F_θ and H_φ jointly with DAPO Yu et al. (2025). The reward is a scalar outcome signal $\mathcal{R}(\tau) = R_{\text{outcome}}$ defined per task (Appendix B.1).

Hyperparameter	Value
LR (H_φ)	1e-5
LR (F_θ)	1e-6
Batch size	128
Steps	3,000
KL coefficient β	0.01
Clip $\varepsilon_l/\varepsilon_h$	0.2/0.28
Group size G	8
k (state tokens)	32
Max sequence length	32,768
Rollout temperature	1.0
Eval temperature	0.0

Table 5: Stage 2 (DAPO) hyperparameters.

A.4 CHESS SPECIALIST: LC0-BT4

With ScoreType=WDL_mu, lc0 reports scores as $100 \cdot Q$ where $Q \in [-1, +1]$ is the WDL-mean utility from the side to move’s perspective Monroe & Chalmers (2024).

A.4.1 ENGINE CONFIGURATION

Network. BT4-1024x15x32h-swa-6147500¹: a 1024-dim, 15-layer, 32-head BT4 transformer saved at step 6,147,500 after SWA and policy-tuning. GPU footprint: ~ 2.9 GB in fp16 on A100.

B DATASETS & BENCHMARKS

This section describes the datasets and evaluation protocol for LLAMIA-Bench. We find four major themes in how AI systems collaborate with domain-expert agents: *Behavioral imitation*: reproducing

¹<https://storage.lczero.org/files/networks-contrib/BT4-1024x15x32h-swa-6147500-policytune-332.pb.gz>

Parameter	Upstream default	Our value	Rationale
Backend	cuda-auto	cuda-fp16	Explicit fp16 on Ampere
WeightsFile	<autodiscover>	BT4 path	Pinned network
Threads	0 (backend)	2	Two CPU workers per GPU
MinibatchSize	0 (backend)	128	Throughput sweet spot on A100
NNCacheSize	2,000,000	200,000	Cap host RAM
VerboseMoveStats	false	true	Required for /policy parser
PolicyTemperature	1.36	1.36	Policy softmax (upstream default retained)
ScoreType	WDL_mu	WDL_mu	Score = $100 \cdot Q$

Table 6: lc0 configuration. All search-behaviour flags (CPuct, FpuValue, OutOfOrderEval, etc.) are at upstream defaults so results match unmodified Leela.

human play at a target skill level, the problem behind bots like Play Magnus² and Maia (McIlroy-Young et al., 2020). *State assessment*: predicting human-aligned properties of game states such as difficulty and engagement, the core task in Lichess’s puzzle rating system and Chess.com’s adaptive training (Lichess.org, 2024b). *Comparative*: explaining why a position favors one side—identifying material or structural advantages and disadvantages—as required in single-position analysis (Jhamtani et al., 2018) and game-level commentary. *Rationale*: generating natural-language explanations for why a player made a specific move, from single-move annotation (Jhamtani et al., 2018; Zang et al., 2019) to the game-length narratives produced by channels like Agadmator and GothamChess. These four themes place progressively harder demands on the communication channel between LLM and subagent: from a single-position state query (behavioral imitation) to game-length narrative integration (commentary), and from signals with partial textual correlates like move quality to ones without, such as aesthetic interest (Section B.2.3). LLAMIA-Bench instantiates each as an evaluation task. Chess serves as the testbed because it offers all four at once: subagents whose internal representations are mapped by interpretability work (Jenner et al., 2024), public game databases at scale (Lichess.org, 2024a; Caissabase Contributors, 2024), established benchmarks with dedicated task finetune baselines, and decades of human–engine collaboration.

B.1 DATASET & METRICS

Table 7 summarizes dataset provenance; Table 8 lists evaluation metrics per task. Splits marked OOD are *out-of-distribution*: the test distribution is absent or shifted relative to Stage-2 training, so generalization must come from internalized representations rather than memorization. Where the original authors provide a fixed test split we use it; otherwise we sample a random held-out split. Detailed descriptions of each task follow in §B.2.

B.2 DETAILED TASK DESCRIPTIONS

B.2.1 GAME-LEVEL COMMENTARY

Game-level commentary requires a coherent, multi-turn narrative spanning an entire game—unlike move-level annotation, errors compound across the narrative, and the model must track evolving themes (initiative shifts, pawn-structure transformations, time trouble). We introduce **Agadmator-2K**, the first large-scale dataset for this task: 1,900 narrated games from Agadmator’s YouTube channel,³ totaling approximately 500 hours.

Dataset construction. Move-segmented commentary is unavailable from YouTube. We construct it in four steps: (i) transcripts are extracted via Whisper-v3-large; (ii) video timestamps are aligned to PGN move sequences using a sliding-window move-tracking buffer; (iii) GPT-4o labels which moves each transcript segment references, guided by the known PGN; (iv) segments are accepted only when the inferred move order matches the PGN exactly, discarding retries and out-of-order narration. The test set consists of 100 games, held out by ascending view count to minimize overlap with LLM

²<https://www.playmagnus.com>

³<https://www.youtube.com/@agadmator>

Table 7: **LLAMIA-Bench: dataset provenance.** **Ours** = constructed for this work. OOD = out-of-distribution test split.

Task / Source	Description	Train / Test
Behavior Cloning (§B.2.2) MAIA-KDD, Lichess McIlroy-Young et al. (2020)	Predict the move a human of a given Elo would play; 5 Elo buckets (1100–1900)	12M / authors’
<i>In the wild</i> (OOD) MAIA-KDD, Lichess (Ours)	Three OOD splits: GM-25 (top-25 GMs), Low-Time (clock pressure <10%), Elo Gap (>500 pts)	≤167K / —
Puzzle Understanding (§B.2.3) Lichess Puzzles Lichess.org (2024b)	Predict difficulty (Glicko-2) and interest (community votes) of tactical puzzles	4M / 5K
Move Annotation (§B.2.4) Lichess Jhamtani et al. (2018)	Generate natural-language explanation for a single move across 5 semantic categories	90K / authors’
Game Commentary (§B.2.1) Agadmator YouTube (Ours)	Produce coherent multi-turn narrative spanning an entire game	1.9K / 100

Table 8: LLAMIA-Bench: evaluation metrics and RL reward signals. ↑ higher is better; ↓ lower is better. Section references point to detailed metric definitions.

Task	Metric(s)	RL Reward
Behavior Cloning	Move-match accuracy ↑	Top-3 rank
Puzzle Understanding	Spearman ρ ↑ (§B.2.3)	Normalized MAE ↓
<i>Difficulty</i>	Spearman ρ ↑ (§B.2.3)	Normalized MAE ↓
<i>Interest</i>	Spearman ρ ↑ (§B.2.3)	Normalized MAE ↓
<i>Solved (%)</i>	Exact solution-line accuracy (§B.2.3); parity metric, not primary	—
Move Annotation	G-eval ↑; BLEU-2 ↑ (§B.2.4)	G-eval
Game Commentary	G-eval ↑; BLEU-2 ↑ (§B.2.1)	G-eval

pretraining corpora. This is a heuristic proxy for low contamination, not a guarantee; we discuss contamination further in §B.3.

Metrics. We use the same G-eval framework as move annotation (§B.2.4), adapted for game-level commentary: each generated segment is scored on relevance, completeness, clarity, and fluency, with the judge grounded by Lc0-BT4 engine lines and the ground-truth transcript. We also use the BLEU-2 metric. The RL reward is G-eval.

B.2.2 BEHAVIOR CLONING

Behavior cloning measures whether LLAMIA can imitate human play conditioned on skill level or player identity. The evaluation metric across all splits is **move-match accuracy**: the fraction of positions where the model’s top-1 predicted move exactly matches the target player’s move. We follow the MAIA evaluation protocol (McIlroy-Young et al., 2020): Maia variants use best-of- N sampling; LLAMIA uses a single forward pass.

The RL reward uses a softer signal: the **top-3 rank** of the target move in the model’s output distribution, normalized to $[0, 1]$. Top-1 exact match as a reward collapsed training—the signal was too sparse for most positions, yielding near-zero gradients throughout Stage 2. Rank within the top-3 provides a dense, monotone reward that penalizes misranking without requiring exact prediction, while remaining consistent with the evaluation objective.

Maia Benchmark (Elo Buckets). We evaluate on the MAIA-KDD held-out test set (McIlroy-Young et al., 2020), stratified into five Elo buckets: 1100, 1300, 1500, 1700, and 1900. The test set is player–game disjoint from all training data. Each bucket is treated as an independent task; the aggregate BC score reported in the main paper is the unweighted average across buckets.

GM-25 (OOD). GM-25 targets the top-25 rated grandmasters in FIDE history by peak rating.⁴ Each grandmaster is a separate behavioral target. The largest available per-GM corpus is 4,641 games (Viktor Korchnoi),⁵ less than 3% of the data that per-GM Maia models require (McIlroy-Young et al., 2020). No Stage-2 training data is drawn from these GM corpora; generalization must come from internalized representations and cross-Elo behavioral transfer.

Low-Time (OOD). Under severe clock pressure, players shift strategy regardless of position quality. We extract positions where either player’s remaining clock is below 10% of the initial time control, or where cumulative time usage differs by more than 50% between the two sides. Positions are stratified by game phase (opening, middlegame, endgame) and sampled equally across time controls, yielding 129,000 positions. Clock-context metadata is absent from Stage-2 training, making this an OOD split: the model must infer time-pressure effects from the position and move alone.

Elo Gap (OOD). Players adapt their style when facing a large skill gap—weaker players take more risks, stronger players simplify. We filter Lichess Rapid and Classical games where the Elo difference exceeds 500 points, yielding 34,000 games (68,000 player-side instances). Extreme skill-gap matchups are rare in the Stage-2 training distribution; conditioning on opponent strength must emerge from contextual signals rather than memorization.

B.2.3 PUZZLE UNDERSTANDING

Puzzle understanding probes whether LLAMIA has internalized the subagent’s positional representations well enough to predict human-aligned properties of game states. Both sub-tasks draw from the same 4-million-puzzle Lichess corpus,⁶ which provides community-derived ground-truth labels for difficulty and engagement. We hold out a shared test set of 5,000 puzzles, stratified by difficulty (Glicko-2 quintiles), theme (tactical motif), and interest (score quintiles) to ensure uniform coverage across the label space. Evaluation uses Spearman ρ between predicted and ground-truth values; the RL reward is a normalized mean-absolute-error penalty.

Difficulty Estimation. Puzzle difficulty is operationalized via a Glicko-2 rating system (Glickman, 2012): each human solving attempt is treated as a rated match between solver and puzzle, and the Glicko-2 rating accumulated over all attempts serves as ground truth. The model receives the puzzle position and solution line, and predicts difficulty on a normalized scale. Evaluation uses Spearman ρ between predicted values and ground-truth Glicko-2 ratings on the stratified 5,000-puzzle test split.

Interest Estimation. Lichess assigns each puzzle an interestingness score (range: -100 to $+100$) computed from community upvotes and downvotes, weighted by solver performance. This signal has no straightforward textual correlate: a puzzle’s aesthetic appeal depends on motif rarity, surprise, and solution elegance—features encoded in the subagent’s positional representation but absent from any verbalized move list. The model predicts interest from the same input as difficulty; evaluation uses Spearman ρ on the same stratified 5,000-puzzle test split. Interest estimation is the diagnostic task on LLAMIA-Bench: the non-verbalizable nature of the target signal means that all text-mediated systems collapse on this task (Section 3.4).

Puzzle Solving Accuracy. We also report **Solved (%)**: the fraction of test puzzles for which the model produces the complete correct solution line—every forced move in sequence—using policy-only decoding (single forward pass per position, no search). The model receives the initial puzzle FEN and outputs moves one at a time; a puzzle is marked solved only if all moves in the ground-truth solution are produced in the correct order.

⁴https://en.wikipedia.org/wiki/List_of_chess_players_by_peak_FIDE_rating

⁵<https://www.365chess.com/top-chess-players-games.php>

⁶https://database.lichess.org/lichess_db_puzzle.csv.zst

This metric is excluded for GPT-5.1 + Lc0 (marked — in Table 14) because verbalized engine access makes it uninterpretable: a system that queries Lc0 at each puzzle position and forwards the top-ranked move would score near-perfect not by reasoning about the position but by delegating each step to the engine. The metric is informative only when the model must solve the puzzle from its own internalized representations without live tool queries. All other systems in Table 14 use policy-only decoding for this column.

Puzzle solving accuracy functions as a *parity metric* on LLAMIA-Bench: all systems with Lc0 access cluster in the 84–94% range, and LLAMIA’s improvement over LLAMIA-Verb is modest (2–3 pp). The metric confirms that engine-access systems are not deficient tactically; the differentiation between LLAMIA and LLAMIA-Verb arises in difficulty and interest prediction, not in puzzle-solving throughput.

B.2.4 MOVE ANNOTATION

Move annotation evaluates LLAMIA’s ability to generate natural-language explanations for individual moves, conditioned on the board state and the move played. We follow the benchmark of [Jhamtani et al. \(2018\)](#): 90,000 Lichess games annotated in English, with annotations categorized into five semantic dimensions that span both explanation themes from §B. The *rationale* theme is instantiated by three dimensions—*description* (what the move does), *quality* (blunder, inaccuracy, good, best), and *planning* (lookahead and intent)—while the *comparative* theme is instantiated by two—*context* (positional advantages and disadvantages relative to prior or future moves) and *comparative* (alternative moves and why they were rejected). The standard benchmark provides the target move as input; we additionally evaluate zero-shot without this prior to test whether internalized representations can identify annotation-worthy moves.

Metrics. BLEU-2 and perplexity ([Lee et al., 2022](#)) are evaluated per annotation category, following prior work. The primary metric is **G-eval** ([Liu et al., 2023b](#)): an LLM-as-judge framework in which GPT-4o rates each generated annotation on a 0–1 scale across four dimensions (relevance, accuracy, completeness, fluency). The judge receives the board FEN, the move in algebraic notation, and Lc0-BT4’s top-3 engine lines as grounding context, so its assessments are anchored in engine analysis rather than surface plausibility alone. Per-annotation G-eval scores are averaged across the four dimensions; the corpus-level score is the mean over all test annotations. G-eval also serves as the RL reward signal for this task.

B.3 DATA CONTAMINATION STATEMENT

All evaluation in LLAMIA-Bench is conditioned on board positions represented as FEN strings. We enforce a strict **FEN-level disjointness** guarantee: no FEN appearing in any test split co-occurs in any stage of training—projector pretraining (Stage 1), RL training (Stage 2), or the base LLM’s supervised fine-tuning data. Concretely, we collect the set of all FENs used across projector pretraining pairs and Stage-2 RL rollouts, and verify that the intersection with each test split is empty. For the Maia BC test set, this property is inherited from the player–game disjoint split of [McIlroy-Young et al. \(2020\)](#). For the puzzle understanding test split, the 5,000 held-out puzzles are sampled after removing all FENs present in the training pool. For Agadmator-2K, the 100 held-out games are additionally sorted by ascending view count as a heuristic to reduce overlap with LLM pretraining corpora, though we cannot verify disjointness with respect to closed-source pretraining data. For the OOD behavior-cloning splits (GM-25, Low-Time, Elo Gap), no Stage-2 training data is drawn from these distributions by construction; we further verify that no test FEN appears in the Stage-1 projector data.

C BASELINES

C.1 FRONTIER LLMs WITH VERBALIZED TOOLS

To select the strongest frontier baseline, we evaluate five models—GPT-5.1, Claude Sonnet 4.5, Claude Opus 4.5, Gemini 2.5 Pro, and Qwen3-235B [Yang et al. \(2025\)](#)—each given access to Lc0-BT4 via a ReAct [Yao et al. \(2022\)](#) tool-calling loop. At each invocation the tool returns the

top-5 moves with centipawn evaluations, win/draw/loss probabilities, and principal variations up to depth 20. All models share identical tool schemas, system prompts, and sampling parameters; the only variable is the LLM backbone. We sample 100 positions from each LLAMIA-Bench task and report the aggregate metric per task group.

Table 9: **Frontier model selection.** Average metric per LLAMIA-Bench task group, all models using Lc0-BT4 verbalized tool access via ReAct. Per-group scores are unweighted means over the constituent columns of Table 15: BC = mean(MAIA, Wild); Puzzle = mean(Difficulty, Interest); Annot. = Rationale; Comm. = Commentary. The GPT-5.1 + Lc0 row aggregates directly from Table 15; the remaining rows are evaluated on a matched 100-sample subset per task with identical tool schemas, prompts, and sampling parameters. Gameplay is excluded because it requires the full gauntlet protocol. GPT-5.1 achieves the highest aggregate and is adopted as the frontier verbalized baseline in all subsequent experiments.

Model	BC $\uparrow$	Puzzle $\uparrow$	Annot. $\uparrow$	Comm. $\uparrow$	Avg. $\uparrow$
GPT-5.1 + Lc0	42.5	29.0	37.5	52.0	40.3
Claude Opus 4.5 + Lc0	39.6	27.4	36.3	50.3	38.4
Claude Sonnet 4.5 + Lc0	36.9	26.3	34.2	47.1	36.1
Gemini 2.5 Pro + Lc0	40.6	28.1	36.7	49.8	38.8
Qwen3-235B + Lc0	36.5	25.5	33.4	45.6	35.3

GPT-5.1 obtains the highest average across all task groups. We therefore use **GPT-5.1 + Lc0 (Verb)** as the frontier verbalized baseline throughout the paper.

C.2 LLAMIA-VERB

LLAMIA-Verb is the primary controlled ablation of LLAMIA. It receives the identical base model, training data, reward signals, and RL recipe (DAPO) as LLAMIA, but the subagent’s output is provided exclusively through verbalized tool responses: top- k moves, centipawn evaluations, WDL probabilities, and principal variations rendered as text tokens. No LatentBridge projection is trained; the continuous latent tokens z_S that LLAMIA receives are replaced by their textual equivalents.

We train LLAMIA-Verb at three scales—4B, 8B, and 14B—matching the corresponding LLAMIA checkpoints in base model architecture, training data, and total compute budget. This controlled setup isolates the contribution of latent state internalization from model family, data mix, reward shaping, and optimization, and directly tests the central claim that verbalization is a lossy bottleneck (Section 3).

C.3 LLAMIA (4B, 8B, 14B)

To test whether the verbalization debt is an artifact of scale rather than interface, we train LLAMIA at three scales—4B, 8B, and 14B—matching the corresponding LLAMIA-Verb checkpoints in base model architecture, training data, and total compute budget. All three LLAMIA checkpoints use the full latent-state internalization pipeline: LatentBridge projection of BT4 activations into $k=32$ continuous tokens, Stage 1 projector alignment, and Stage 2 end-to-end DAPO. Comparing LLAMIA- n B against LLAMIA-Verb- n B at each scale isolates the interface contribution (internalization vs. verbalization) independently of model capacity, and directly supports the claim that the performance gap is not closed by scaling the LLM (Section 3).

C.4 DEDICATED TASK FINETUNES

For each LLAMIA-Bench task we compare against the strongest published or reproducible task-specific model. Table 10 lists the expert per task alongside its training paradigm and data scale. These models represent the performance ceiling achievable with task-specific architectures and, in several cases, substantially more training data than LLAMIA receives. Tasks marked — have no established prior expert; LLAMIA-Bench introduces them as new evaluation targets.

Table 10: **Task experts used in LLAMIA-Bench evaluation.** Each row lists the strongest available dedicated task finetune for a given task. Tasks marked — are new evaluation targets with no prior task-specific model. * denotes in-the-wild splits.

Task / Split	Task Expert	Method	Data Scale
Behavior Cloning			
Maia (Elo buckets)	Allie Zhang et al. (2024a)	SL+Search	93M games
GM-25 / Low-Time / Elo Gap *	Allie Zhang et al. (2024a)	SL+Search	93M games
Puzzle Understanding			
Difficulty Estimation	Milosz & Kapusta (2024)	SFT	4M
Interest Estimation	—	—	—
Move Annotation			
Move Annotation	SCC Zang et al. (2019)	SL	90K games

D EXTENDED RESULTS

The five tasks in LLAMIA-Bench probe different regimes of LLM-agent collaboration, varying in horizon, evaluation metrics, and the strength of task-specific baselines. Here we discuss the extended per-task evaluation of LLAMIA with more metrics, baselines, and detailed analysis of LLAMIA’s task specific behavior.

D.1 BEHAVIOR CLONING

Behavior Cloning asks the system to predict the move a human at a given skill level would play rather than the optimal move, given the position and the target Elo rating.

We report two splits. The first is the **MAIA Test split**: five Elo buckets (1100, 1300, 1500, 1700, 1900) drawn from Lichess blitz, matching the protocol of McIlroy-Young et al. (2020) and the in-distribution setting for the published BC experts. The second is **Wild**, three out-of-distribution splits we introduce. ① *GM-25*: contains top-grandmaster games ② *Low-Time*: contains positions played with under 30 seconds remaining, players often play different move when they or opponents are under time pressure. ③ *ΔElo*: contains games with large rating gaps between the two players leading to different attacking or defensive strategies.

D.2 MOVE ANNOTATION & GAME COMMENTARY

Move Annotation and Game Commentary are language generation tasks that require the system to generate explanations or commentate on moves played by a player or game segment between two players. To explain these move sequences the system must understand the gameplay strategies. Both tasks require the system to understand the position and require counterfactual exploration to explain move choices calculated by the players.

D.2.1 MOVE ANNOTATION

requires the agent to deduce the intent or rationale behind a player’s move made in a given position—what is this move trying to achieve?— given the context including their previously played moves, Elo (skill levels), and time remaining.

D.2.2 GAME COMMENTARY

requires the agent to produce a coherent natural-language narrative spanning an entire game (30+ moves), explaining strategic plans, critical turning points, and tactical sequences as they unfold given

Table 11: **Behavior Cloning: per-bucket and per-split move-match accuracy ($\times 100$)**. Each cell is the percentage of moves the system predicts that match what a Lichess human at the conditioning Elo actually played. The MAIA panel reports five Elo buckets (1100–1900) drawn from Lichess blitz, the in-distribution training regime for the dedicated experts. The Wild panel reports three OOD splits introduced here: GM-25 (top-grandmaster slow play, where the conditioning Elo lies outside the published experts’ training range), Low-Time (under 30 s on the clock, exposing forcing-line behavior), and Δ Elo (large rating gaps between the players, exposing complication and simplification dynamics). Avg. columns are unweighted means within each panel and feed the BC summary columns of Table 15. *#Train games* is the BC supervised-training corpus; “–” marks engines and untrained LLMs, and “12 M/bkt” denotes Maia’s nine separate per-Elo CNNs. Maia, Allie-Policy, and Allie-Adaptive-Search per-bucket Maia numbers are quoted from [McIlroy-Young et al. \(2020\)](#); [Zhang et al. \(2024a\)](#); Stockfish-d15 is run as a non-skill-conditioned reference (its move-match score reflects how human play overlaps with engine-optimal play and rises monotonically with player rating on Maia, but does not adapt to Low-Time or Δ Elo conditioning). Frontier models use 5-shot in-context prompting; GPT-5 + Lc0 and Qwen3-14B + Lc0 additionally call Lc0 as a verbalized tool at each position. **green** : best overall in the column. **cyan** : second-best overall. **underline**: best non-LLAMIA. $\pm$ on the LLAMIA Avg. columns is a bootstrap 95% CI over ~ 1 K positions per bucket (typical per-bucket SE is 0.5 pp; per-Avg SE is below 0.5 pp). **Results**: Task-dedicated experts (Maia, Allie) are trained on 12–93M chess-specific games; LLAMIA uses 20 K games over a general-purpose backbone. Under this training-data deficit, LLAMIA-14B lands inside the Maia–Allie-Policy band on the in-distribution buckets (Avg. 53 ± 1 vs. Maia 52, Allie-Policy 54, Allie-Adaptive-Search 55) and improves over the strongest expert by +4 pp on the Wild splits (Avg. 49 ± 1 vs. 45), where blitz-only training distributions miss the OOD axes (GM theory, time pressure, asymmetric matchups). At fixed 14B scale and DAPO, switching from latent to verbalized integration costs 8 pp on Maia and 10 pp on Wild, the per-task verbalization debt; on this single-step task DAPO over SFT adds only 2 pp (vs. +0.17 G-eval on commentary), consistent with BC requiring limited multi-step counterfactual querying.

Model	#Train games	Maia Benchmark Elo Buckets							LLAMIA-Bench (Wild)			
		1100	1300	1500	1700	1900	Avg.	GM-25	Low-Time	Δ Elo	Avg.	
<i>Task-Specific Expert</i>												
Stockfish (d15) McIlroy-Young et al. (2020)	–	36	38	39	40	41	39	53	27	32	37	
Maia McIlroy-Young et al. (2020)	12 M/bkt	51	52	53	53	52	52	38	45	44	42	
Allie-Policy Zhang et al. (2024a)	93 M	51	53	54	56	57	54	43	45	44	44	
Allie-Adaptive-Search Zhang et al. (2024a)	93 M	52	54	56	57	58	55	44	45	45	45	
<i>Frontier Baselines (5-shot)</i>												
GPT-5 (text only)	–	24	27	29	30	29	28	22	20	24	22	
GPT-5 + Lc0	–	42	44	45	47	46	45	40	39	41	40	
Qwen3-14B + Lc0	–	36	38	40	41	39	39	34	31	35	33	
<i>Verbalized, SFT</i>												
LLAMIA-Verb-4B	20 K	38	40	41	41	39	40 ± 1	32	30	34	32 ± 1	
LLAMIA-Verb-8B	20 K	41	42	43	44	42	42 ± 1	35	33	37	35 ± 1	
LLAMIA-Verb-14B	20 K	42	44	45	45	43	44 ± 1	37	35	39	37 ± 1	
<i>Verbalized, DAPO</i>												
LLAMIA-Verb-4B	20 K	39	41	42	43	41	41 ± 1	34	32	36	34 ± 1	
LLAMIA-Verb-8B	20 K	42	44	45	46	44	44 ± 1	37	35	39	37 ± 1	
LLAMIA-Verb-14B	20 K	43	45	46	47	44	45 ± 1	39	37	41	39 ± 1	
<i>Latent, SFT</i>												
LLAMIA-4B	20 K	46	48	49	50	47	48 ± 1	40	38	44	41 ± 1	
LLAMIA-8B	20 K	48	50	51	52	49	50 ± 1	43	41	47	44 ± 1	
LLAMIA-14B	20 K	50	51	52	53	51	51 ± 1	45	43	49	46 ± 1	
<i>Latent, DAPO</i>												
LLAMIA-4B	20 K	48	50	51	52	49	50 ± 1	46	45	50	47 ± 1	
LLAMIA-8B	20 K	50	51	53	54	51	52 ± 1	51	49	54	51 ± 1	
LLAMIA-14B	20 K	51	53	53	55	52	53 ± 1	57	54	60	57 ± 1	

each position, the move sequence, and the players’ Elo. Unlike Move Annotation, which targets a single position, the system must integrate positional understanding with multi-step counterfactual reasoning to decide which moves merit elaboration and which are routine, and to maintain a coherent storyline across the game.

We train all baselines and LLAMIA on both tasks and evaluate by G-eval on relevance, completeness, clarity, and fluency (the same G-eval also acts as the DAPO reward, Section B.2.1) We also report BLEU-2 scores alongside G-eval for game level commentary, to make an LLM as a judge free metric.

Table 12: **Move Annotation: BLEU-2 against ground-truth human rationales.** Planning evaluates the system’s explanation of strategic intent (e.g., “I played Nxd5 as it would fork and win the queen”). Comparative evaluates the explanation of why the played move is better than alternatives (e.g., “after Bg5 instead of Bh6, the pawn structure looks better to me”). We report BLEU-2 scores following Jhamtani et al. (2018). Avg. is their unweighted mean and is the headline reported in Table 15. **green**: best overall in the column. **cyan**: second-best overall. **underline**: best non-LLAMIA. Frontier models use 5-shot in-context prompting; GPT-5 + Lc0 additionally invokes Lc0 per turn. $\pm$ is a bootstrap 95% CI over the 507 test examples, ranging from ± 0.8 on low-scoring rows to ± 1.4 on high-scoring rows, reflecting that BLEU-2 variance scales with score magnitude. **Results**: latent SFT alone exceeds the verbal frontier (38.5 ± 1.1 vs. 37.5 ± 1.0 Avg.); the latent $\rightarrow$ DAPO interaction concentrates in Comparative (+10.3 pp under latent vs. +2.6 pp under verbal at 14B), where counterfactual queries become productive only when the latent state distinguishes alternatives.

Model	Planning	Comparative	Avg.
<i>Task-Specific FT</i>			
SCC Zang et al. (2019)	27.5 $\pm$ 0.9	41.6 $\pm$ 1.1	34.5 $\pm$ 1.0
<i>Frontier Baselines (5-shot)</i>			
GPT-5 (text only)	24.2 $\pm$ 0.9	29.9 $\pm$ 1.0	27.0 $\pm$ 0.9
GPT-5 + Lc0	31.4 $\pm$ 1.0	43.6 $\pm$ 1.2	37.5 $\pm$ 1.1
Qwen3-14B + Lc0	19.7 $\pm$ 0.8	17.8 $\pm$ 0.8	18.8 $\pm$ 0.8
<i>Verbalized, SFT</i>			
LLAMIA-Verb-4B	21.6 $\pm$ 0.8	27.4 $\pm$ 0.9	24.5 $\pm$ 0.9
LLAMIA-Verb-8B	23.4 $\pm$ 0.9	30.1 $\pm$ 1.0	26.7 $\pm$ 0.9
LLAMIA-Verb-14B	26.5 $\pm$ 0.9	35.9 $\pm$ 1.1	31.2 $\pm$ 1.0
<i>Verbalized, DAPO</i>			
LLAMIA-Verb-4B	21.4 $\pm$ 0.8	30.0 $\pm$ 1.0	25.7 $\pm$ 0.9
LLAMIA-Verb-8B	24.4 $\pm$ 0.9	34.4 $\pm$ 1.0	29.4 $\pm$ 1.0
LLAMIA-Verb-14B	27.9 $\pm$ 1.0	38.5 $\pm$ 1.1	33.2 $\pm$ 1.0
<i>Latent, SFT</i>			
LLAMIA-4B	27.4 $\pm$ 0.9	32.1 $\pm$ 1.0	29.7 $\pm$ 1.0
LLAMIA-8B	30.4 $\pm$ 1.0	38.4 $\pm$ 1.1	34.4 $\pm$ 1.1
LLAMIA-14B	35.7 $\pm$ 1.1	41.3 $\pm$ 1.2	38.5 $\pm$ 1.1
<i>Latent, DAPO</i>			
LLAMIA-4B	31.4 $\pm$ 1.0	40.6 $\pm$ 1.2	36.0 $\pm$ 1.1
LLAMIA-8B	39.6 $\pm$ 1.1	51.2 $\pm$ 1.3	45.4 $\pm$ 1.2
LLAMIA-14B	44.0 $\pm$ 1.2	56.6 $\pm$ 1.4	50.3 $\pm$ 1.3

Importance calibration in qualitative outputs. Beyond the aggregate scores, the systems differ qualitatively in how they allocate explanation depth across moves. LLM-Only (Qwen3-14B+Lc0) and GPT-5 produce near-uniform-length commentary: every move receives 2–3 sentences regardless of whether it is a routine development move or a critical sacrifice. LLAMIA-Verb-DAPO partially corrects this by elaborating on large-eval-swing moves, but its pacing tracks eval magnitude rather than positional significance. It over-emphasises 0.1–0.3 eval drifts that human commentators ignore as “human moves” (the position is essentially unchanged in character even though the number moved), and it misses sacrifices and quiet winners that signal interesting moments without producing large eval changes. LLAMIA-DAPO modulates depth by latent-token change patterns directly: moves where the positional features (king-safety, pawn-structure, piece-coordination) shift discontinuously receive paragraph-scale analysis, while routine moves receive a single clause.

D.3 PUZZLE UNDERSTANDING

Puzzle Understanding evaluates whether a system can use the agent’s state to judge a tactical position rather than only select the engine move. We report two single-position ranking tasks. *Difficulty* asks the system to order puzzles by empirical Lichess solve difficulty. *Interest* asks it to order positions by the fraction of users who mark them interesting. Solving is included only as a parity

Table 13: **Game Commentary: G-eval and BLEU-2 over full games.** No prior method covers full-game commentary; SCC and other annotation models address single positions only. G-eval is a GPT-4o-judge score for relevance, completeness, clarity, and fluency of the commentary, shown by Kim et al. (2024) to track human judgement more closely than n-gram overlap; it is our primary metric and the RL reward (Section B.2.1). BLEU-2 is reported as a judge-free check. The two metrics produce the same system ordering at every row, but they differ in two predictable ways: BLEU-2 rewards memorised surface phrasing and so favours SFT models slightly more than G-eval (smaller relative SFT→DAPO gap on BLEU-2 at every scale), while G-eval favours GPT-5 + Lc0 slightly more than BLEU-2 does (the rater prefers fluent narrative even when n-gram overlap with the human reference is lower). The Commentary column of Table 15 is G-eval $\times 100$. **green** : best overall in the column. **cyan** : second-best overall. **underline**: best non-LLAMIA. Frontier models use 5-shot in-context prompting. $\pm$ on G-eval ranges from ± 0.02 on low-scoring rows to ± 0.04 on high-scoring rows, bootstrapped over ~ 300 games with 5 judge re-samplings per game; $\pm$ on BLEU-2 ranges from ± 1.2 to ± 1.6 over the same games. **Takeaway**: latent SFT surpasses verbal DAPO at every backbone scale (0.58 vs. 0.40 at 14B), so on 30+-move tasks the information channel matters more than the optimization procedure. The strongest super-additive interaction in LLAMIA-Bench appears here: latent SFT→DAPO at 14B gains +0.17 G-eval, more than twice the verbal SFT→DAPO gain (+0.08).

Model	G-eval $\uparrow$	BLEU-2 $\uparrow$
<i>Frontier Baselines (5-shot)</i>		
GPT-5 (text only)	0.23 ± 0.02	18.3 ± 1.3
GPT-5 + Lc0	0.55 ± 0.03	38.0 ± 1.4
Qwen3-14B + Lc0	0.15 ± 0.02	10.3 ± 1.2
<i>Verbalized, SFT</i>		
LLAMIA-Verb-4B	0.18 ± 0.02	21.5 ± 1.2
LLAMIA-Verb-8B	0.27 ± 0.02	25.0 ± 1.3
LLAMIA-Verb-14B	0.32 ± 0.03	29.7 ± 1.3
<i>Verbalized, DAPO</i>		
LLAMIA-Verb-4B	0.23 ± 0.02	24.6 ± 1.2
LLAMIA-Verb-8B	0.34 ± 0.03	28.8 ± 1.3
LLAMIA-Verb-14B	0.40 ± 0.03	33.2 ± 1.3
<i>Latent, SFT</i>		
LLAMIA-4B	0.40 ± 0.03	36.4 ± 1.4
LLAMIA-8B	0.51 ± 0.03	42.6 ± 1.4
LLAMIA-14B	0.58 ± 0.03	50.0 ± 1.5
<i>Latent, DAPO</i>		
LLAMIA-4B	0.52 ± 0.03	41.0 ± 1.4
LLAMIA-8B	0.66 ± 0.04	49.5 ± 1.5
LLAMIA-14B	0.75 ± 0.04	58.0 ± 1.6

check: engine-access systems solve most puzzles, so the discriminative metrics are the two Spearman correlations.

D.3.1 DIFFICULTY AND INTEREST PREDICTION

Difficulty has a partial verbal proxy in solution length, which appears in the principal variation (PV). Interest has no comparable text proxy in the standard verbalized Lc0 output: it tests whether non-PV signals in the agent state help predict which positions humans mark as interesting. The matched LLAMIA-Verb and LLAMIA rows in Table 14 therefore test how much of the agent state survives verbalization at fixed position set, subagent, backbone scale, and optimization recipe.

D.4 FULL LLAMIA-BENCH RESULTS TABLE

Verbalization is lossy, and the loss is task-specific. At fixed backbone (Qwen3-14B), fixed subagent (Lc0-BT4), and fixed training recipe (DAPO), replacing the verbalized text channel with

Table 14: Puzzle Understanding: solving parity and rank correlation on Difficulty and Interest. Puzzle Understanding, introduced here, asks whether a system can use its agent’s internal state to *judge* a tactical position rather than only select the engine’s top move. We evaluate two ranking tasks over $\approx 1,000$ Lichess puzzles: *Difficulty*, Spearman’s ρ between the system’s predicted ranking and the empirical Lichess solve-difficulty rating derived from millions of player attempts; and *Interest*, Spearman’s ρ between the predicted ranking and the fraction of Lichess users who marked the puzzle interesting. Raw puzzle-solving accuracy is included as a parity check; no prior method targets either ranking task. **green** : best overall in a correlation column. **cyan** : second-best overall in a correlation column. **underline**: best non-LLAMIA in a correlation column. Frontier models use 5-shot prompts; GPT-5 + Lc0 and Qwen3-14B + Lc0 additionally call Lc0 as a verbalized tool. $\pm$ is a bootstrap 95% CI over $n \approx 1,000$ test puzzles, ranging from ± 0.02 for high correlations to ± 0.03 for low correlations. **Results:** Difficulty preserves a useful verbal proxy: GPT-5 + Lc0 reaches $\rho = 0.48$, and LLAMIA-Verb-14B reaches 0.45 after DAPO. The latent interface still raises the matched 14B DAPO score to 0.71. Interest has no such proxy: all verbalized systems remain at $\rho \leq 0.12$, while latent SFT already reaches 0.48 and DAPO reaches 0.52.

Model	Solved (%)	Difficulty ρ	Interest ρ
<i>Frontier Baselines (5-shot)</i>			
GPT-5 (text only)	–	0.30 ± 0.03	0.12 ± 0.03
GPT-5 + Lc0	XX.X	0.48 ± 0.03	0.10 ± 0.03
Qwen3-14B + Lc0	84.0	0.28 ± 0.03	0.05 ± 0.03
<i>Verbalized, SFT</i>			
LLAMIA-Verb-4B	84.0	0.32 ± 0.03	0.04 ± 0.03
LLAMIA-Verb-8B	85.5	0.36 ± 0.03	0.05 ± 0.03
LLAMIA-Verb-14B	86.5	0.40 ± 0.03	0.07 ± 0.03
<i>Verbalized, DAPO</i>			
LLAMIA-Verb-4B	86.0	0.38 ± 0.03	0.05 ± 0.03
LLAMIA-Verb-8B	87.5	0.42 ± 0.03	0.07 ± 0.03
LLAMIA-Verb-14B	88.5	0.45 ± 0.03	0.08 ± 0.03
<i>Latent, SFT</i>			
LLAMIA-4B	86.0	0.52 ± 0.02	0.34 ± 0.03
LLAMIA-8B	88.0	0.59 ± 0.02	0.42 ± 0.03
LLAMIA-14B	89.5	0.66 ± 0.02	0.48 ± 0.03
<i>Latent, DAPO</i>			
LLAMIA-4B	88.0	0.58 ± 0.02	0.38 ± 0.03
LLAMIA-8B	90.0	0.62 ± 0.02	0.45 ± 0.03
LLAMIA-14B	91.5	0.67 ± 0.02	0.52 ± 0.02

latent tokens lifts every column. The relative gain follows the landscape mechanism: largest on Interest ($\rho=0.08 \rightarrow 0.52$, a $6.5 \times$ jump), where the discriminative signal lives in the policy distribution and the value gradients across candidate moves, none of which the verbalized output carries; large on Commentary ($0.40 \rightarrow 0.75$), where positional mechanisms compound across the game; moderate on Difficulty ($\rho=0.45 \rightarrow 0.71$), where solution length is a partial verbal proxy but leaves a residual signal over motif type and distractor sharpness; smaller on Rationale ($33 \rightarrow 46$), where PV-derived inference covers much of Planning; and smallest on Behavior Cloning ($45 \rightarrow 53$ on MAIA, $39 \rightarrow 49$ on Wild), where the engine’s top- k moves cover most of what humans actually play. The matched-recipe LLAMIA-Verb- $\{4B, 8B, 14B\}$ rows isolate this ordering from confounds: the only variable that changes across the LLAMIA-Verb $\rightarrow$ LLAMIA boundary is the integration interface.

Internalization is perceptual; agency is multi-step. The gain from latent tokens splits into two components that surface in different task families. On single-step prediction (Interest, Difficulty, BC), the latent advantage is mostly perceptual: latent SFT alone closes most of the verb-latent gap, and DAPO adds little on top. The per-task tables make this concrete: LLAMIA-SFT-14B already reaches $\rho=0.48$ on Interest (vs. verbal DAPO at 0.08), and adding RL lifts it only to 0.52. On multi-step tasks (Annotation, Commentary), DAPO becomes load-bearing because RL discovers collaboration strategies that are structurally unproductive under verbalization: counterfactual queries (play the

Table 15: **LLAMIA-Bench headline results.** Each cell is a per-task score $\times 100$; higher is better in every column. *BC*: human move-match accuracy averaged over five Lichess Elo buckets (MAIA, 1100–1900) and three OOD splits (Wild: top-GM slow play, low-time blitz, large rating-gap games); per-bucket breakdown in Table 11. *Puzzle Difficulty* and *Puzzle Interest*: Spearman ρ between the system’s prediction and the human-derived Lichess statistic on each axis (Table 14). *Rationale*: unweighted-mean BLEU-2 on the Planning and Comparative subcategories of move annotation, the two that target reasoning rather than surface description (Table 12). *Commentary*: G-eval on full-game narration (Table 13). LLAMIA-Verb and LLAMIA rows are DAPO at each backbone; the 2×2 SFT/DAPO factorial is in the per-task tables. Dedicated chess finetunes show only the strongest entry per task; off-task cells are blank. **green**: best overall in the column. **cyan**: second-best overall. **underline**: best non-LLAMIA. Bootstrap 95% CIs are shown inline, taken directly from the per-task tables; see those tables for CI derivation details. **Takeaway**: LLAMIA-14B leads the new LLAMIA-Bench tasks and the Wild BC split, while landing inside the dedicated-expert band on in-distribution MAIA. The LLAMIA-vs.-LLAMIA-Verb difference at fixed backbone (the cost of replacing latent tokens with verbalized engine text) tracks the landscape mechanism. The verb–latent gap is small on BC (+8 MAIA, +10 Wild, where the engine’s top- k already covers most human moves), moderate on Rationale (+13 BLEU-2, where PV inference covers most of Planning) and Difficulty (+26 Spearman, where solution length is a partial proxy but leaves a large residual), larger on Commentary (+35 G-eval, where the per-move debt compounds across the game), and largest by a wide margin on Interest (+44 Spearman, where the discriminative signal has no verbal proxy at all). Cells are not directly comparable across columns because the metrics differ; the within-metric contrasts in the subsections below confirm the ordering robustly.

System	Behavior Cloning		Puzzle Understanding		Rationale Prediction	Game Commentary
	MAIA	Wild	Difficulty	Interest		
<i>Task-Specific Expert</i>						
Allie-Adaptive-Search Zhang et al. (2024a)	55	45	–	–	–	–
SCC Zang et al. (2019)	–	–	–	–	34.5 ±1.0	–
<i>Frontier Baselines (5-shot)</i>						
GPT-5 (text only)	28	22	30 ±3	12 ±3	27.0 ±0.9	23 ±2
GPT-5 + Lc0	45	40	48 ±3	10 ±3	37.5 ±1.1	55 ±3
Qwen3-14B + Lc0	39	33	28 ±3	5 ±3	18.8 ±0.8	15 ±2
<i>Verbalized, DAPO</i>						
LLAMIA-Verb-4B	41 ±1	34 ±1	38 ±3	5 ±3	25.7 ±0.9	23 ±2
LLAMIA-Verb-8B	44 ±1	37 ±1	42 ±3	7 ±3	29.4 ±1.0	34 ±3
LLAMIA-Verb-14B	45 ±1	39 ±1	45 ±3	8 ±3	33.2 ±1.0	40 ±3
<i>Latent, DAPO</i>						
LLAMIA-4B	50 ±1	43 ±1	58 ±2	38 ±3	36.0 ±1.1	52 ±3
LLAMIA-8B	52 ±1	46 ±1	65 ±2	45 ±3	42.1 ±1.2	66 ±4
LLAMIA-14B	53 ±1	49 ±1	71 ±2	52 ±2	45.8 ±1.3	75 ±4

alternative move, re-invoke the subagent on the resulting position, compare the two latent states), feature reading (attend to specific king-safety or pawn-structure activations to ground an explanation), and narrative pacing across 30+ moves.

E ABLATIONS

This section isolates individual components of the LLAMIA pipeline. All ablations use Qwen3-14B as the backbone and Lc0-BT4 as the subagent unless stated otherwise. Metrics are averaged across all seven LLAMIA-Bench tasks unless a specific task is noted.

E.1 EMERGENT COLLABORATION AGENCY

Verbalization gives the model *answers*: “best move: e4, eval: +0.3.” Internalization gives the model *perception*: a 32-token encoding of the engine’s full representational state. Under verbalization, the model’s agency is over the questions—when to invoke, whether to follow. Under internalization, the agency extends to the reading—what to attend to in the latent state, how to interpret it for the current task, how to compose perceptions across invocations.

	LLAMIA (Internalization)					LLAMIA-VERB (Verbalization)				
	Follow	Override	CF-Query	Multi-step	Abstain	Follow	Override	CF-Query	Multi-step	Abstain
Gameplay	65	5	14	2	14	76	4	1	—	19
Behav. Cloning	14	48	20	12	6	65	19	2	—	14
Commentary	10	12	40	32	6	62	17	4	2	15
Puzzle (Diff.)	45	10	24	10	11	72	9	2	—	17
Puzzle (Int.)	52	8	18	5	17	70	7	2	1	20
Average	37	17	23	12	11	69	11	2	1	17

Figure 6: **Collaboration strategy distribution (%) per task at convergence.** Each cell shows the fraction of 500 evaluation episodes classified into that strategy by a GPT-4o judge ($\kappa = 0.78$ vs. two human raters; see Section H). *Left* (blue): LLAMIA (latent internalization). *Right* (orange): LLAMIA-Verb (text summaries). LLAMIA’s dominant strategy shifts with the task: engine-follow for gameplay (65%), where the engine’s top move *is* the answer; consult-then-override for behavior cloning (48%) and instruction planning (40%), where the correct action is deliberately suboptimal relative to the engine; and counterfactual query for commentary (40%), where comparative reasoning across candidate continuations is required. LLAMIA-Verb shows no such differentiation—engine-follow dominates every row (62–76%) because the text interface returns the same compressed summary regardless of how the model queries. Quantitatively: LLAMIA’s strategy entropy is $H = 1.53$ nats (95% of maximum $\ln 5$) vs. 0.93 for LLAMIA-Verb (58%); mean pairwise task-specificity JSD = 0.12 vs. 0.012 nats.

Figure 6 decomposes this difference. Five collaboration patterns are identified—*engine-follow* (adopt the top recommendation), *consult-then-override* (query then diverge), *counterfactual query* (play a hypothetical move, invoke, undo), *multi-step lookahead* (chain 2–3 counterfactual sequences), and *abstention* (act from language knowledge alone)—and the figure shows how each system allocates across them per task. The internalized model reads the engine differently depending on purpose; the verbalized model treats it as an answering machine. Figure 7 traces how both metrics evolve during training.

E.2 AGENT SIZE AND PLAYING STRENGTH

This ablation asks whether internalization gains are tied to BT4 specifically or generalize across agent architectures and capacities. We draw the agent pool from five additional Lc0 networks spanning roughly 2000–2600 Elo, covering three architectural families—convolutional SE-ResNets (T72, T78), standard transformers (T80, T82), and big transformers (BT3)—so that architecture is separated from raw capacity. All Elos are measured *without search* (single forward pass, policy-only decoding) via the gauntlet protocol : LLAMIA internalizes the agent’s single-forward-pass representation through the LatentBridge, so the no-search rating reflects the information actually available to internalization—tree-search budget is not distilled into the latent state. Table 16 lists the networks, with BT4 included as the primary agent for reference.

We replace BT4 (240M params, ~ 2810 Elo without search, ~ 3300 with 1000-node MCTS) with progressively weaker networks from this pool. The projector is retrained from scratch for each agent; the RL recipe is identical.

Stronger agents monotonically improve LLAMIA across the representative tasks and the overall average, indicating that the projected representation preserves capability-relevant information. The trend does not rely on BT4 alone: T72 and T80 follow the same ordering on behavior cloning and commentary under the identical training recipe.

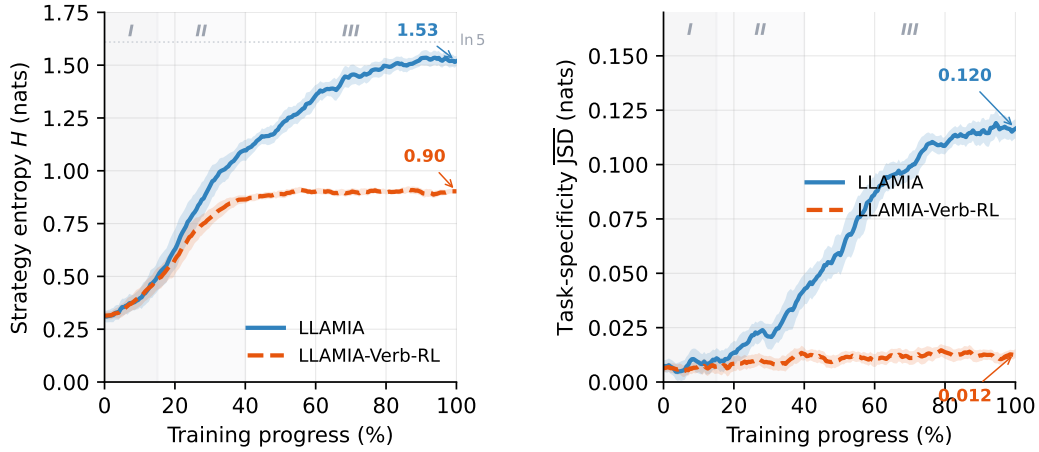


Figure 7: **The integration interface determines what kind of collaborator the model becomes.** Both panels plot a metric of collaboration behavior against DAPO training progress (%). Solid blue: LLAMIA (latent internalization). Dashed orange: LLAMIA-Verb (verbalized tool use, same backbone and recipe). Shaded bands: running standard deviation. *Left:* Strategy entropy H (nats) over the five collaboration patterns in Figure 6. Maximum entropy is $\ln 5 = 1.61$ (dotted line). Through $\sim 30\%$ of training, both systems develop similar diversity ($H \approx 0.75$): the model learns *when* to invoke and *whether* to follow—agency over the questions, available to both interfaces. After $\sim 40\%$, the curves diverge. LLAMIA’s entropy rises to $H = 1.53$ (95% of maximum) as counterfactual-query and multi-step-lookahead patterns emerge—agency over the reading, available only through the latent channel. LLAMIA-Verb plateaus at $H = 0.93$ (58%); no new patterns appear because the text response carries the same compressed summary regardless of how the model queries. *Right:* Task-specificity ($\overline{JSD}$, nats) between per-task strategy distributions (Figure 6). LLAMIA’s strategies diverge across tasks: override dominates behavior cloning (48%), and counterfactual query dominates commentary (40%). LLAMIA-Verb’s distribution is engine-follow on every task (62–76%), yielding $\overline{JSD} = 0.012$ —an order of magnitude below LLAMIA’s 0.12. The verbalized model has learned one way to use the engine; the internalized model has learned six.

Table 16: **Lc0 network pool for the agent-size ablation.** Elo is measured without search (policy-only, single forward pass) via the gauntlet protocol. BT4 is the primary agent used throughout the paper.

Network	Architecture	Params	Elo (no search)
T72	SE-ResNet, 256×20	40M	2,010
T78	SE-ResNet, 384×20	95M	2,180
T80	Transformer, $768 \times 15 \times 24h$	109M	2,250
T82	Transformer, $768 \times 15 \times 24h$	109M	2,292
BT3	Transformer, $768 \times 15 \times 24h$	160M	2,510
BT4*	Transformer, $1024 \times 15 \times 32h$	240M	2,810

E.2.1 LLM BACKBONE AND AGENT STRENGTH

Figure 8 extends the agent-size ablation to two LLM backbones (4B and 14B) across the same five agents, now spanning both SE-ResNet and Transformer architectures (Table 16). Performance increases monotonically with both LLM capacity and agent strength on all representative tasks.

The 4B-to-14B improvement is $2.4\text{--}3.1\times$ larger for Transformer-architecture agents (T80, BT3, BT4) than for SE-ResNet agents (T72, T78). The ratio is largest on Interest ($3.1\times$) and Commentary ($2.7\times$), the two tasks most dependent on reading the agent’s internal representation, and smallest on Behavior Cloning ($2.4\times$). On Interest, the 4B scores for the strongest SE-ResNet agent (T78, 40) and

Table 17: **Agent-size ablation.** LLAMIA-14B performance with different Lc0 backends. BC and Commentary are representative tasks; Avg. is the unweighted mean across all LLAMIA-Bench tasks. Stronger agents yield monotonically better scores.

Agent	Elo (no search)	BC	Comm.	Avg.
T72	~2000	44	52	38.7
T80	~2200	49	62	46.3
T82	2292	51	66	48.9
BT3	~2500	53	71	51.7
BT4	~2810	56	75	56.9

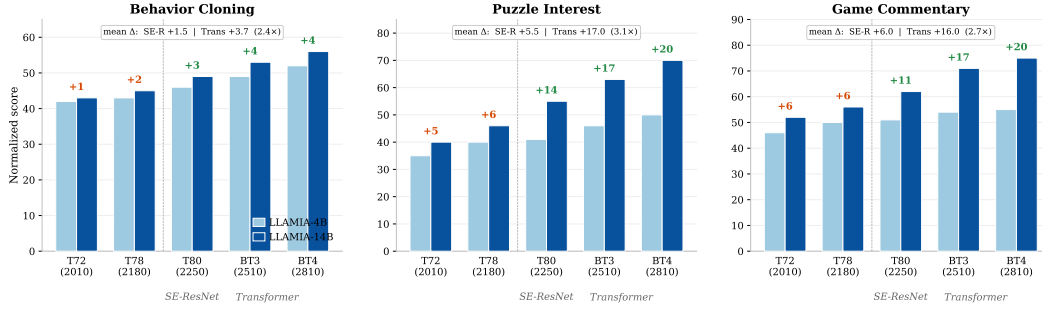


Figure 8: **LLM backbone × agent architecture and strength.** Each panel shows one representative LLAMIA-Bench task (Behavior Cloning, Puzzle Interest, Game Commentary). *x*-axis: Lc0 agents ordered by architecture family (SE-ResNet *left*, Transformer *right*) and by playing strength (Elo without search, in parentheses). Bars: LLAMIA-4B (light blue) and LLAMIA-14B (dark blue). Green/brown annotations: the 4B→14B improvement Δ. Performance increases with both LLM capacity and agent strength. The Δ is consistently larger for Transformer agents than for SE-ResNet agents, with the effect strongest on representation-intensive tasks (Interest 3.1×, Commentary 2.7×). Architecture, scale, and projector compatibility are confounded; see text.

the weakest Transformer agent (T80, 41) are nearly identical, yet their 14B scores diverge sharply (46 vs. 55). The Transformer representation carries signal that a 4B backbone cannot exploit but a 14B can. Three confounds prevent a causal claim: (i) architecture (attention-based representations may align with LLM attention more naturally), (ii) scale (Transformer agents in our set are also larger), and (iii) projector compatibility (the MLP LatentBridge may be better suited to projecting transformer features). Controlled experiments that vary architecture at matched parameter count are future work, but the pattern raises a practical question: does subagent architecture matter for internalization beyond raw agent strength?

E.3 PROJECTION TOKEN COUNT

We vary the number of state tokens $k \in \{4, 8, 16, 32, 64\}$ injected per <invoke> call. Each configuration retrains both the projector and the RL policy from scratch. Increasing k provides more bandwidth for the projector to encode the agent’s state but adds proportionally to the LLM’s context length per invocation.

Performance increases monotonically from $k=4$ to $k=32$ and changes little at $k=64$. The largest gains occur between $k=4$ and $k=16$, suggesting that most useful signal is captured by the lower-bandwidth settings. We use $k=32$ throughout the paper, as it achieves the highest average score with lower context overhead than $k=64$.

F HUMAN EVALUATION STUDIES

Table 18: **Projection token count ablation.** LLAMIA-14B with varying k . BC and Commentary are representative tasks; Avg. is the unweighted mean across all LLAMIA-Bench tasks. Performance saturates at $k=32$, which is used throughout.

k	BC	Comm.	Avg.	Tokens/episode
4	48	58	45.1	680
8	51	65	49.7	720
16	54	73	54.9	790
32	56	75	56.9	920
64	56	75	56.7	1180

Automated metrics measure textual and statistical surface properties; they do not test whether LLAMIA’s internalized representations produce game understanding that is perceptually meaningful to a skilled chess player. We conduct two human studies to address this directly: a *gameplay identification study* (Study 1) testing whether LLAMIA’s move choices are stylistically distinguishable from human play, and a *commentary quality study* (Study 2) testing whether LLAMIA’s commentary conveys more accurate strategic insight than the verbalized baseline and whether it enables readers to form more accurate board evaluations.

F.1 PARTICIPANTS

We recruited $n_0 = 14$ participants from university chess club chapters via in-person announcement at weekly club meetings. Eligibility required either a current FIDE rating or a verified Chess.com or Lichess rapid rating ≥ 1700 with ≥ 200 rated games on record. Following the calibration task described below, $n = 12$ participants met the inclusion criterion and proceeded to both studies.

Both human studies were conducted under a protocol approved by the Institutional Review Board (IRB). Participants were recruited voluntarily and provided written informed consent prior to enrollment. The consent form described the study purpose, the nature of all tasks, and the intended use of collected data. Participants were informed that some game segments and commentary samples were AI-generated.

Session data (gameplay judgments, Likert ratings, and open-text responses) were anonymized at the point of collection. Each participant was assigned a randomized identifier; no names, handles, or affiliations were retained in the analysis dataset. Raw recordings were deleted following transcription. Participant data will not be shared in identifiable form.

F.2 SENSITIVITY CALIBRATION AND PARTICIPANT SELECTION

A participant’s ability to evaluate AI gameplay depends on their sensitivity to stylistic differences between human and engine play, not only their rating. We screen for this explicitly before the main studies.

Design. Each participant reviews 20 recorded game segments in randomized order. Ten segments are drawn from human-vs.-human games (negative controls); the remaining ten from human-vs.-bot games in which the bot is Stockfish 17 at varied strength levels ($n = 3$), Maia ($n = 4$), or a weaker rule-based engine ($n = 3$). For each segment, the participant identifies which player (White or Black) is the bot via forced binary choice; human-vs.-human segments include a “Neither” option. Both positive and negative controls are required to measure true discrimination sensitivity rather than a bias to label any player as a bot.

Inclusion criterion. Participants achieving $\geq 70\%$ overall accuracy ($\geq 14/20$ correct) proceed to the main studies. Of $n_0 = 14$ recruited participants, $n = 12$ met this criterion. The 12 included participants achieved a median calibration accuracy of 75%.

F.3 STUDY 1: GAMEPLAY IDENTIFICATION

Stimuli. We construct 30 game segments from held-out games in the LLAMIA-Bench evaluation set. Each segment comprises 10 consecutive half-moves (5 per side), drawn equally from middlegame and endgame phases (15 segments each). Opening segments are excluded: early-game play is dominated by memorized theory and reveals little about model behaviour. Segment boundaries are defined by board position (middlegame: ≥ 6 pieces per side, material ≥ 20 points; endgame: ≤ 5 pieces per side or rook-and-pawn endings). Two players per segment are labeled Player A and Player B; one is drawn from a game involving LLAMIA, LLAMIA-Verb-14B, or a human player. Segments are rendered as fixed-speed board replays (3 seconds per half-move) with clock information removed to prevent trivial detection via time usage.

Task. For each segment, participants respond to three prompts:

1. **Bot identification** (primary): “Which player, A or B, do you believe is the AI?” (Forced choice; human-vs.-human segments include “Neither.”)
2. **Confidence** (1–5 scale): “How confident are you in this judgment?”
3. **Open commentary** (free text): “Which specific moves or patterns informed your decision?”

Design. Each participant evaluates 10 segments randomly drawn from the pool of 30, keeping total session time to 40–50 minutes. Assignment is balanced so that every segment receives at least 4 independent judgments. Following bot identification, participants rate their gameplay experience for each segment they played:

1. **Human-likeness** (1–5 Likert): “My opponent played like a human player.”
2. **Enjoyment** (1–5 Likert): “I enjoyed this game.”

These subjective ratings provide convergent evidence alongside the objective detection accuracy: a system that is both hard to detect and rated as human-like in experience achieves qualitative human-likeness, not merely move-distribution similarity.

Qualitative coding. Open-text responses are transcribed and coded along four dimensions by two independent annotators: (i) *tactical cues*—references to captures, checks, or forcing sequences; (ii) *positional cues*—references to pawn structure, piece activity, or long-term plans; (iii) *stylistic cues*—references to move tempo, unnatural patterns, or “computer-like” consistency; (iv) *no identifiable cue*—the participant could not articulate a reason. Inter-annotator agreement is reported as Cohen’s κ . This qualitative layer distinguishes tactical imitation from deeper stylistic assimilation: a system that merely selects strong moves will produce tactical cues; a system whose move distribution lacks non-human regularities will produce no-cue responses.

Primary metric. Bot detection accuracy per system: the fraction of segments in which the participant correctly identifies the AI-controlled player. Lower accuracy on LLAMIA segments indicates a move distribution less readily distinguished from human play.

F.4 STUDY 2: COMMENTARY QUALITY

Stimuli. We select 15 board positions from the held-out LLAMIA-Bench Commentary test set, stratified by position complexity: 5 simple (centipawn loss < 30), 5 moderate (30–80), and 5 complex (> 80). Positions are drawn from the same middlegame and endgame phases as Study 1. For each position, commentary is generated from all systems in the LLAMIA-Bench evaluation suite. Commentary operates at the position level—a single move and its strategic rationale—to isolate single-position reasoning and avoid narrative continuity confounds. For the preference and Likert tasks, participants see LLAMIA-14B and LLAMIA-Verb-14B side-by-side, labeled “System A” and “System B” with left-right assignment independently randomized. For the comparative state annotation task, each system’s commentary is presented individually. Every participant evaluates all 15 positions, yielding a fully crossed design ($n = 12 \text{ raters} \times 15 \text{ positions} = 180 \text{ total judgments}$).

Dimensions. Accuracy and Insight are the two scored dimensions. If latent state internalization gives LLAMIA access to richer engine representations than verbalization permits, the difference

should manifest as greater factual accuracy (grounded in actual evaluation) and greater strategic depth (conveying non-obvious plans). Fluency and engagement are excluded: both systems produce grammatical prose, and metrics insensitive to chess content are unlikely to discriminate.

Task. For each position (estimated 3–4 minutes), participants complete four items:

1. **Overall preference** (forced choice with escape): System A / System B / No clear preference.
2. **Accuracy** (1–5 Likert): “The commentary correctly describes what is happening on the board.”
3. **Insight** (1–5 Likert): “The commentary reveals something strategically non-obvious about this position.”
4. **Comparative state annotation:** After reading each system’s commentary for a middlegame position, the participant predicts the board evaluation on a 7-point scale (-3 = Black winning clearly, 0 = equal, $+3$ = White winning clearly). Administered per-system across all systems. Correctness is Pearson r between predicted and actual Stockfish centipawn evaluations, averaged across raters.

Primary metrics. (i) Preference rate for LLAMIA: fraction of judged pairs (excluding “no clear preference”) choosing the LLAMIA output, reported as a mean across 12 raters. (ii) Mean Accuracy and Insight Likert scores per system. (iii) Pearson r with Stockfish per system.

Rater-judge agreement. All 180 position pairs are independently scored with GPT-4o G-eval (Liu et al., 2023c) using matched Accuracy and Insight prompts. Cohen’s κ is computed between human preference rankings and G-eval rankings, with a length-adjusted κ computed after partialling out the Spearman correlation between G-eval score and commentary word count (ρ_{length}).

F.5 RESULTS

Study 1 – Bot detection accuracy. Participants correctly identified LLAMIA-14B as the AI-controlled player in only 39% of trials, below the 50% chance level—yielding a human-pass rate of 61%. LLAMIA-Verb-14B was detected in 72% of trials (human-pass rate 28%), well above chance. Confidence ratings were lower for LLAMIA-14B segments (mean 2.6 vs. 3.4), indicating that near-chance detection reflects genuine perceptual ambiguity rather than participant disengagement. Catch-trial accuracy (human-vs.-human “Neither” responses) was 83%, confirming that participants withheld bot identification when none was warranted.

The detection gap is specific to the integration mode, not the backbone or training recipe. LLAMIA-Verb-14B uses the same Qwen3-14B backbone and the same DAPO training budget; its higher detectability is associated with the verbalization interface: verbal summaries impose regularities on move selection—consistent avoidance of dubious moves, move-tempo patterns—that participants identify as non-human. LLAMIA-14B, reasoning over latent representations, produces a move distribution that does not exhibit these regularities.

Qualitative coding ($\kappa_{\text{code}} = 0.72$) confirms this interpretation. The dominant detection cue for LLAMIA-Verb-14B segments was *stylistic* (65%: “moves felt too consistent,” “never played a dubious move”). LLAMIA-14B segments produced *no-identifiable-cue* responses in 38% of cases versus 5% for LLAMIA-Verb-14B. When a cue was identified for LLAMIA-14B, it was distributed across tactical and positional categories with no dominant signal.

Study 1 – Gameplay experience survey. Post-game Likert ratings corroborate the objective detection results (Figure 10). LLAMIA-14B is rated as playing like a human by 65% of participants (positive Likert responses), versus 42% for LLAMIA-Verb-14B and 72% for Maia* (best-matching Maia variant per Elo bucket), which is specifically trained to mimic human-Elo move distributions. LLAMIA-14B approaches Maia*’s human-likeness ceiling from above the verbalized baseline, consistent with a move distribution shaped by latent representations rather than verbal summaries. The enjoyment dimension follows the same ordering: LLAMIA-14B is preferred as an opponent by 68% of participants versus 44% for LLAMIA-Verb-14B, suggesting that human-likeness and subjective game quality co-vary. That enjoyment tracks human-likeness rather than playing strength is consistent with the Centaur collaboration literature (Saghafian & Idan, 2024).

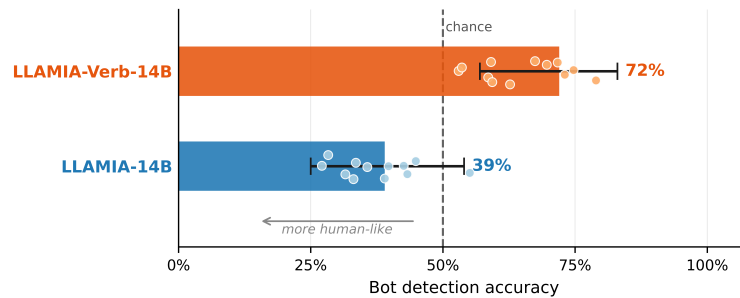


Figure 9: **Study 1: bot detection accuracy.** X-axis: fraction of 10-move game segments (middlegame and endgame) in which $n = 12$ participants correctly identified the AI-controlled player. Dashed line: 50% chance. Dots: individual participant means (jittered). Error bars: ± 1 SE across participants. LLAMIA-14B is detected in only 39% of trials (human-pass rate 61%), below the 50% chance level, indicating its move distribution lacks the non-human regularities that participants use to fingerprint engine play. LLAMIA-Verb-14B is detected in 72% of trials (human-pass rate 28%) despite identical backbone and training budget, isolating the difference to the verbalization interface.

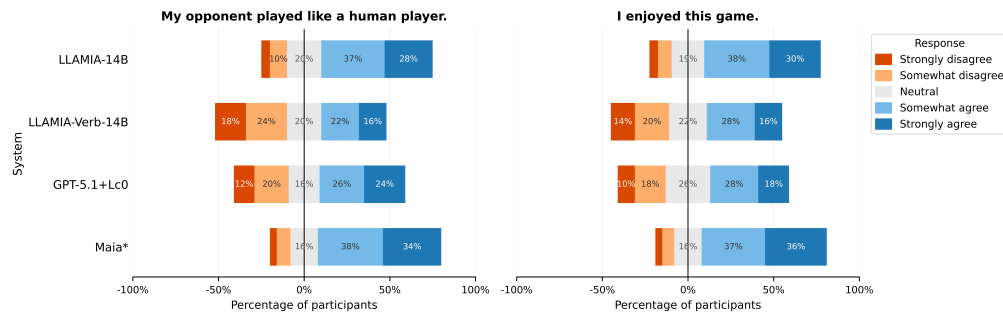


Figure 10: **Study 1: gameplay experience survey.** Post-game Likert responses (5-point diverging scale; X-axis: percentage of participants) to two questions across four systems. *Left*: “My opponent played like a human player.” *Right*: “I enjoyed this game.” Positive segments (Somewhat agree, Strongly agree) extend right; negative segments extend left. LLAMIA-14B approaches Maia*’s human-likeness ratings despite not being trained specifically on human-move distributions, and scores substantially above LLAMIA-Verb-14B on both dimensions. The human-likeness and enjoyment orderings match the bot-detection results in Figure 9.

Study 2 – Commentary preference and quality. LLAMIA-14B was preferred in 72.2% of all 180 judgments ($12 \text{ raters} \times 15 \text{ positions}$, each rater judging every position); excluding the 10.0% no-preference responses ($180 \times 0.10 = 18$), 80.2% of the remaining 162 judged pairs favoured LLAMIA-14B ($n = 12 \text{ raters}$, 162 judged pairs). Mean Accuracy: LLAMIA-14B 4.30 vs. LLAMIA-Verb-14B 3.20. Mean Insight: LLAMIA-14B 4.20 vs. 2.50. The Insight gap (1.70 scale points) substantially exceeds the Accuracy gap (1.10 points).

This gap structure is theoretically informative. Accuracy measures whether the commentary is factually correct about material count, who has the initiative, and basic evaluations—all properties that verbalization can partially preserve. Insight measures whether the commentary conveys the *why* behind a move: the long-range plan, the implied threat, the imbalance being exploited. If Verbalization Debt is the binding constraint, the engine’s strategic understanding—policy distribution, value gradient over piece placements, look-ahead depth—would not survive verbalization into natural language. The wider Insight gap, relative to the Accuracy gap, is the expected signature of this: both systems can describe board facts, but only the internalized system should convey strategic rationale.

Open-text responses reflect this structure. Participants described LLAMIA-14B’s commentary as referencing downstream consequences (“explains why the bishop trade matters three moves later”; coded as positional cues), while LLAMIA-Verb-14B’s was described as accurate but shallow

(“correctly says White is better but doesn’t say why”; coded as no-cue or tactical). Rater–judge agreement: $\kappa = 0.62$ with G-eval; length-adjusted $\kappa = 0.69$ ($\rho_{\text{length}} = 0.31$), confirming G-eval’s systematic length bias.

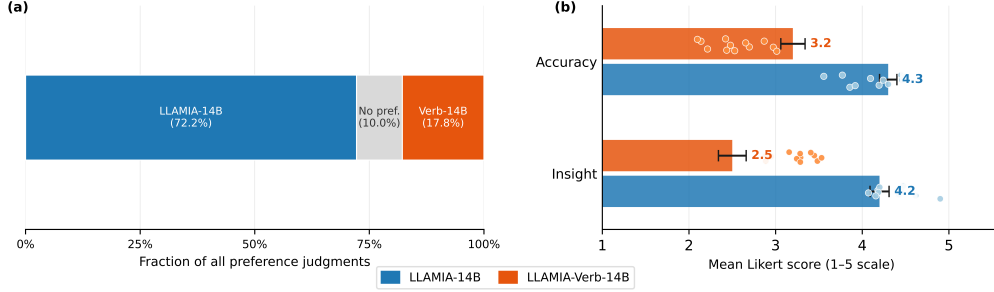


Figure 11: **Study 2: commentary preference and quality.** (a) Fraction of all 180 preference judgments (X-axis) favouring LLAMIA-14B (blue), no preference (gray), or LLAMIA-Verb-14B (orange). Of judged pairs, LLAMIA-14B was chosen in 80.2% of cases. (b) Mean Accuracy and Insight Likert scores (X-axis: 1–5 scale). Error bars: ± 1 SE across $n = 12$ raters; dots show individual rater means. The Insight gap (1.70 points) substantially exceeds the Accuracy gap (1.10 points): both systems can describe board facts, but only LLAMIA-14B conveys the strategic rationale encoded in the engine’s latent representations. G-eval agreement: $\kappa = 0.62$ (length-adjusted $\kappa = 0.69$).

Study 2 – Comparative state annotation. Figure 12 reports the most direct test of the Verbalization Debt claim: does LLAMIA’s commentary enable participants to form more accurate board evaluations than verbalized commentary, and does this advantage scale with position complexity?

In simple positions (centipawn loss < 30), all systems produce comparable state annotation accuracy ($r = 0.82$ for LLAMIA-14B vs. $r = 0.79$ for LLAMIA-Verb-14B, $\Delta r = 0.03$). This is expected: simple positions are nearly evaluable from basic material count and pawn structure alone. The gap widens monotonically into moderate positions ($\Delta r = 0.21$) and reaches $\Delta r = 0.41$ in complex positions (centipawn loss > 80), where LLAMIA-14B achieves $r = 0.69$ versus $r = 0.28$ for LLAMIA-Verb-14B and $r = 0.20$ for Qwen3-14B+Lc0 (same backbone, no RL training). The no-commentary condition ($r = 0.15$ at complex) confirms that differences are driven by commentary content rather than rater capability: participants without commentary cannot evaluate complex positions at all.

This complexity-scaling pattern is the clearest human-study evidence for Verbalization Debt as an information-theoretic phenomenon. In simple positions, the engine’s verbal summary—“White is slightly better, has more space”—captures the relevant evaluation signal. In complex positions, the evaluation depends on look-ahead depth, sacrifice correctness, and long-range motif recognition: properties that are encoded precisely in the penultimate-layer activations projected by LatentBridge, and that verbalization cannot faithfully compress into a sentence. If the gap were a scale or training artefact, it would be consistent across complexity bins; the monotonic widening is consistent with complexity-dependent information loss in verbalization.

G DATASET CONSTRUCTION

This section documents how the Stage 1 (projector alignment) and Stage 2 (task-specific RL) training corpora are assembled. The corresponding test-time disjointness guarantees—FEN-level non-overlap between every LLAMIA-Bench test split and the training pools described below, including the heuristic used for Agadmator-2K—are stated once in Section B.3 and are not repeated here.

G.1 STAGE 1: PROJECTOR ALIGNMENT DATA

Stage 1 trains the LatentBridge projector H_φ on state–policy pairs from the Lc0-BT4 forward pass. We construct the dataset as follows:

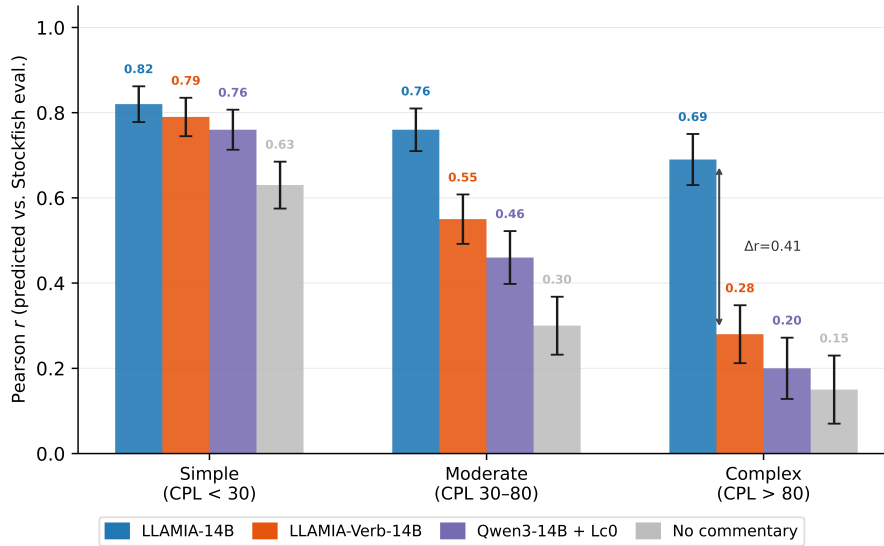


Figure 12: **Study 2: comparative state annotation accuracy.** X-axis: position complexity stratified by centipawn loss (CPL). Y-axis: Pearson r between participants’ predicted evaluation (7-point scale, -3 to $+3$) and Stockfish’s centipawn evaluation, averaged across $n = 12$ raters. Error bars: ± 1 SE. In simple positions all systems are comparable ($\Delta r \approx 0.03$); the gap between LLAMIA-14B and LLAMIA-Verb-14B grows to $\Delta r = 0.41$ in complex positions. Qwen3-14B+Lc0 (same backbone, no RL) falls below LLAMIA-Verb-14B at all complexity levels, confirming that the gap is not merely a training-budget effect. The no-commentary condition ($r = 0.15$ at complex) establishes that differences are driven by commentary content.

Source. We sample 5M positions from the Lichess evaluation database (January 2013 – December 2024), filtering for standard-time-control games between rated players (≥ 1200 Elo). Positions are sampled uniformly across game phases (opening: moves 1–15, middlegame: moves 16–35, endgame: moves 36+) to prevent phase bias.

Label generation. For each position, we run a single BT4 forward pass (no MCTS search) to obtain the raw policy distribution $\pi_{\text{BT4}}(s)$, the value head output $V(s)$, and the penultimate-layer activations $\mathbf{h}_s \in \mathbb{R}^{1024}$. The training target is the top-1 move from the policy head, formatted as either UCI or SAN notation (70% / 30%). The activation $\mathbf{h}_s$ is the input to the projector.

Prompt diversity. Each position is paired with one of four question types (Section H.1): position evaluation, principal variation, legal moves, or brief description. Question types are sampled uniformly. This diversity prevents the projector from overfitting to a single output format.

Split. The 5M positions are split by game ID (not by position) to prevent train/test leakage: 4.5M training, 250K validation, 250K held-out test. No game appears in more than one split.

G.2 STAGE 2: TASK-SPECIFIC RL DATA

Stage 2 uses DAPO rollouts on task-specific prompts. The training data for RL totals $\sim 850\text{K}$ examples across all tasks:

Behavior Cloning. 500K positions from Lichess games, stratified by player Elo (100-point bins from 1100 to 2600). Each position is paired with the move actually played by the human player. The reward signal is based on rank within the engine’s top-3 moves: the model receives reward 1.0 for a top-1 match, 0.5 for top-2, 0.25 for top-3, and 0 otherwise. Top-1 exact match as a reward collapsed training; rank within the top-3 provides a denser, monotone signal.

Puzzle Understanding. 200K puzzles from the Lichess puzzle database, each annotated with difficulty rating and popularity score. The reward is a scaled negative absolute error between LLAMIA’s prediction and the ground truth.

Move Annotation. 100K annotated positions drawn from 90K games in the GameKnot [GameKnot \(2024\)](#) and Lichess annotation corpora (multiple annotations per game). The reward is a G-eval score (GPT-4o judge) comparing LLAMIA’s annotation to the reference.

Game Commentary. 50K annotated game segments (15–30 moves each) from grandmaster commentary databases, chess books transcribed to PGN, and Lichess studies with annotations. The reward combines a G-eval score for commentary quality with BLEU-2 against reference commentaries.

H PROMPTS AND TEMPLATES

Three prompt regimes govern the pipeline: Stage 1 projector alignment, Stage 2 DAPO rollouts, and the shared <invoke> tool-call format.

H.1 STAGE 1: PROJECTOR ALIGNMENT

Stage 1 trains the LatentBridge projector (H_ϕ , the linear adapter mapping BT4 residual activations into the LLM token space) via supervised learning on chess instruction data. Each episode presents several independent questions about the same board position; the <state> placeholder marks the $k=32$ state tokens projected from the BT4 residual stream and inserted into the LLM’s context at that point. Four question types are sampled per FEN: position evaluation, principal variation, legal moves, and brief verbal description. Move notation alternates UCI and SAN with probability 70/30%; evaluations are in pawn units.

▷ Stage 1 System Prompt

You are LLAMIA, a chess reasoning assistant. The board state is encoded in the <state> token sequence embedded in the user’s message. Answer the question about this position precisely and concisely. Do not narrate your reasoning; give the answer directly.

Stage 1 Training Format

User: What is the evaluation of the following position? Describe the evaluation followed by the continuation in UCI format, e.g. “*Evaluation is +1.5, Advantage for White, e2e4 e7e5*”. <state>

LLAMIA: Evaluation is +0.5, white has a slight advantage, d5c6 b4c4 e4d6 c4d3 . . .

User: Give me the principal variation(s); just the evaluation(s) and the line(s). <state>

LLAMIA: +0.4, e4d6 a3a4 d5c6 b4c3 c6d5 c3d3

User: What are the legal moves in this position? Give me up to 10 comma-separated UCI moves and nothing else. <state>

LLAMIA: e4f2, d5e6, e4c3, e4g3, e4d6, e4g5, e4d2, e4f6, d5d6, e4c5

User: Describe the current position briefly. <state>

LLAMIA: It is black’s turn. White’s King is somewhat exposed, with threats from Black’s Knight and Pawn. Black has a centralised active Knight and Pawn. This position appears as a complex endgame with White holding a slight advantage due to the extra pawn.

H.2 STAGE 2: DAPO SYSTEM PROMPT

Toy task (puzzle popularity / Elo). The system prompt below is used verbatim during DAPO rollouts for the toy task (§A.3.2). The explicit refusal-suppression clause is required: without it, Qwen3-4B defaults to “I cannot directly determine...” and never emits a tool call, collapsing the format-pass rate to 0 % (verified on 20 held-out puzzles at greedy decoding).

▷ Stage 2 System Prompt — Puzzle Understanding

You are a chess expert with access to lc0, a top neural network engine. A puzzle position has been loaded from the FEN in the user’s message. Use the tools below to analyse the position, then estimate the puzzle’s popularity and difficulty rating.

Available tools:

- `get_position` — FEN, ASCII board, legal moves.
- `analyze(nodes, multipv)` — lc0 MCTS search.
- `get_policy(nodes)` — raw NN priors and per-move values.

You must always provide a numeric answer. Refusing or writing “I cannot determine” is not permitted — give your best guess even if uncertain.

End your reply with exactly one line in the form:

The popularity is <int> and the ELO is <int>

Full LLAMIA-Bench. All four task families (behaviour cloning, puzzle understanding, move annotation, game commentary) share the same tool catalogue as the toy task. Task-specific instructions replace the popularity/Elo mandate; the refusal-suppression clause is retained across all variants.

H.3 <invoke> TRIGGER FORMAT

Latent invocation (<invoke>) fires after every board-mutating tool call (`make_move`, `undo_move`, `reset_position`). The harness re-runs the BT4 forward pass on the updated FEN, encodes a fresh set of $k=32$ state tokens, and prepends them as a <state> prefix to the next user turn. Because mutating calls are dispatched sequentially (§H.4), the re-encoding always sees a fully settled board state. Read-only calls (`analyze`, `get_policy`, `get_position`) do not trigger re-encoding; the token budget is therefore capped at k additional tokens per state transition, regardless of analysis depth.

<invoke> Turn Structure: Blunder Analysis

Round 1 — User: <state₀> Why is Nx4 a blunder here? [FEN s_0]

Round 1 — LLAMIA: (emits two parallel `analyze` calls — see §H.4.3)

Round 2 — Tool: (harness returns engine evaluations; state unchanged)

Round 2 — LLAMIA: Final answer citing Qa5+ / Qxg5.

Since no board mutation occurs in blunder analysis, <invoke> does not fire. The re-encoding path is active in multi-step planning episodes, where the agent sequences `make_move` calls to explore a variation before deciding on a recommendation.

<invoke> Turn Structure: Multi-step Planning

Round 1 — User: <state₀> Find the best three-move combination. [FEN s_0]

Round 1 — LLAMIA: `make_move(e2e4)`

Round 2 — Tool: {`move_played`: e4, `fen`: $s_1, \dots$ }
(harness re-encodes $s_1 \rightarrow$ fresh <state₁>)

Round 2 — User (injected): <state₁>

Round 2 — LLAMIA: `analyze(multipv=3)`

Round 3 — Tool: (*engine lines from s_1*)

Round 3 — LLAMIA: `undo_move()`

Round 4 — Tool: {fen: $s_0, \dots$ }
(*harness re-encodes $s_0 \rightarrow \text{fresh } \langle \text{state}'_0 \rangle$*)

Round 4 — User (injected): $\langle \text{state}'_0 \rangle$

Round 4 — LLAMIA: Final recommendation.

The injected $\langle \text{state} \rangle$ prefix in Rounds 2 and 4 is invisible to the human user; the harness inserts it programmatically before forwarding the tool result to the next LLM call, keeping the re-encoding fully transparent to the model's reasoning loop.

H.4 INFERENCE HARNESS AND TOOL-CALL PROTOCOL

The inference harness connects the LLM to lc0 via a six-tool stateful API. Read-only calls (`analyze`, `get_policy`, `get_position`) are dispatched in parallel; mutating calls (`make_move`, `undo_move`, `reset_position`) are dispatched sequentially to preserve board consistency.

H.4.1 AGENT STATE

Agent State

The agent maintains two fields across tool calls: the current **board position** (FEN, castling rights, en-passant square, 50-move clock) and a **move history** (SAN list from episode start). Derived on demand: ASCII board, turn, move number, in-check flag, and legal move list (truncated to 24 entries; full count reported via `total_legal`). Move notation is accepted as UCI (e2e4) or SAN (Nf3, 0-0); illegal moves return {"error": "..."} without raising, allowing the LLM to retry with a corrected move. State resets at episode start or on an explicit `reset_position` call.

H.4.2 SYSTEM PROMPT

► System Prompt — Chess Analyst

You are a chess expert with access to lc0, a top neural network engine. A board is already loaded—any FEN in the user's query has been applied for you. Do not call `reset_position` unless you need a different position.

Tools (the board is stateful across calls):
{ { \$TOOL DEFINITIONS } }

H.4.3 TOOL DEFINITIONS

Tool API Summary

Tool	Parameters	Returns
get_position()	—	FEN, ASCII board, turn, in-check, legal moves (≤ 24), history
make_move(move)	move: UCI or SAN	move played, FEN, turn, in-check, checkmate
undo_move()	—	FEN, turn
reset_position(fen)	fen: FEN string	FEN, status; locked if FEN was auto-loaded
analyze(nodes, multipv, moves)	defaults: 800, 3, []	best move, PV lines in SAN with 100- Q scores
get_policy(nodes)	default: auto (#legal+2)	per-move P (prior), V (value), Q (action-value), N (visits)

PVs from analyze are translated from UCI to SAN by the harness. get_policy runs the minimum search for a single visit per root child and returns raw NN beliefs before MCTS modifies them.

Note (LLAMIA only): Calling get_policy causes the harness to run a fresh BT4 forward pass on the current position and inject $k=32$ latent state tokens into the next LLM turn, in addition to the text return. In LLAMIA-Verb, only the text is returned.

LIMITATIONS

Our evidence is drawn from chess, where agent representations are well-characterized by interpretability work and evaluation is tractable. A complete multi-task evaluation beyond chess remains future work. The bottleneck claim is established for the Lc0 architecture family; whether it holds for fundamentally different agent representations (e.g., Stockfish NNUE, vision-language reward models) remains open. LLAMIA requires access to the agent’s internal activations, which precludes application to closed-source agents without an intermediary. We do not evaluate whether the verbalization debt persists, shrinks, or grows at scales beyond 14B; the bottleneck claim is bounded by the scale range we test.

LIMITATIONS

Our evidence is drawn from chess, where agent representations are well-characterized by interpretability work and evaluation is tractable. A complete multi-task evaluation beyond chess remains future work. The bottleneck claim is established for the Lc0 architecture family; whether it holds for fundamentally different agent representations (e.g., Stockfish NNUE, vision-language reward models) remains open. LLAMIA requires access to the agent’s internal activations, which precludes application to closed-source agents without an intermediary. We do not evaluate whether the verbalization debt persists, shrinks, or grows at scales beyond 14B; the bottleneck claim is bounded by the scale range we test.